Unit-5
Service Layer Protocols and Security

- Services Layer
  - One M2M
  - ETSI M2M
  - OMA
  - BBF

- Security in IoT Protocols
  - MAC 802.15.4
  - 6LoWPAN
  - RPL
  - Application Layer

Why Do We Need a Service Layer?

1. **Fragmentation**
   Each vertical (health, energy, transport) built its own stack and APIs.

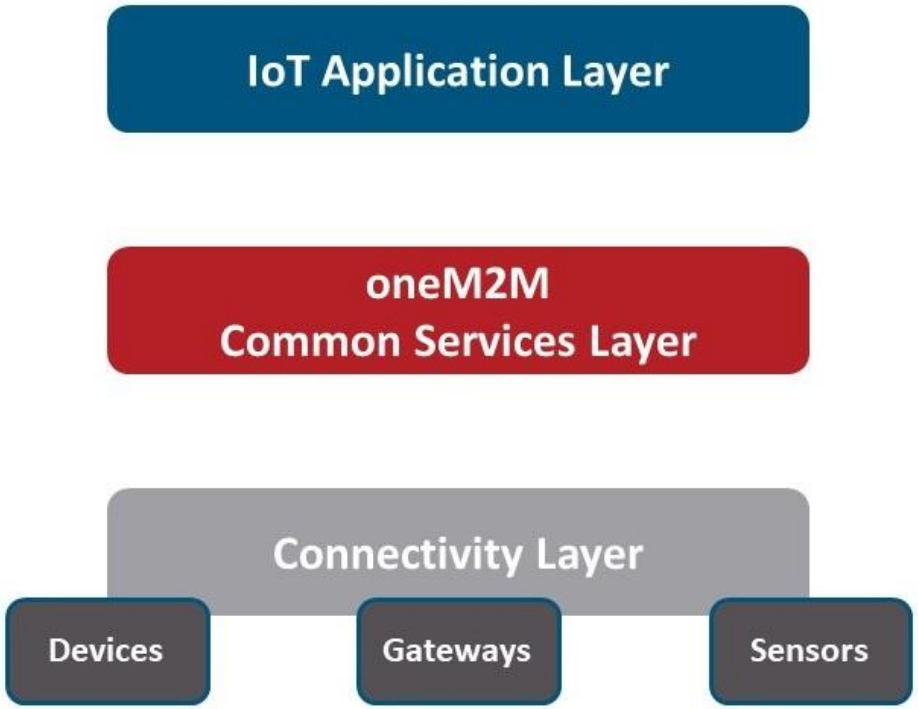
2. **Duplicated effort**
   Registration, security, data storage and device management re-implemented every time.

3. **No interoperability**
   Applications cannot reuse devices or data across domains and vendors.

Solution: a common horizontal layer between applications and networks.

What is oneM2M?

- Global standard for an M2
- Developed by 8 standards TTC, TTA, CCSA, TSDSI
- Horizontal, access-networ
- Exposes services through
- Embedded in devices, gat



2012
Founded

8
SDO partners

2015
Release 1

Services Layer-One M2M

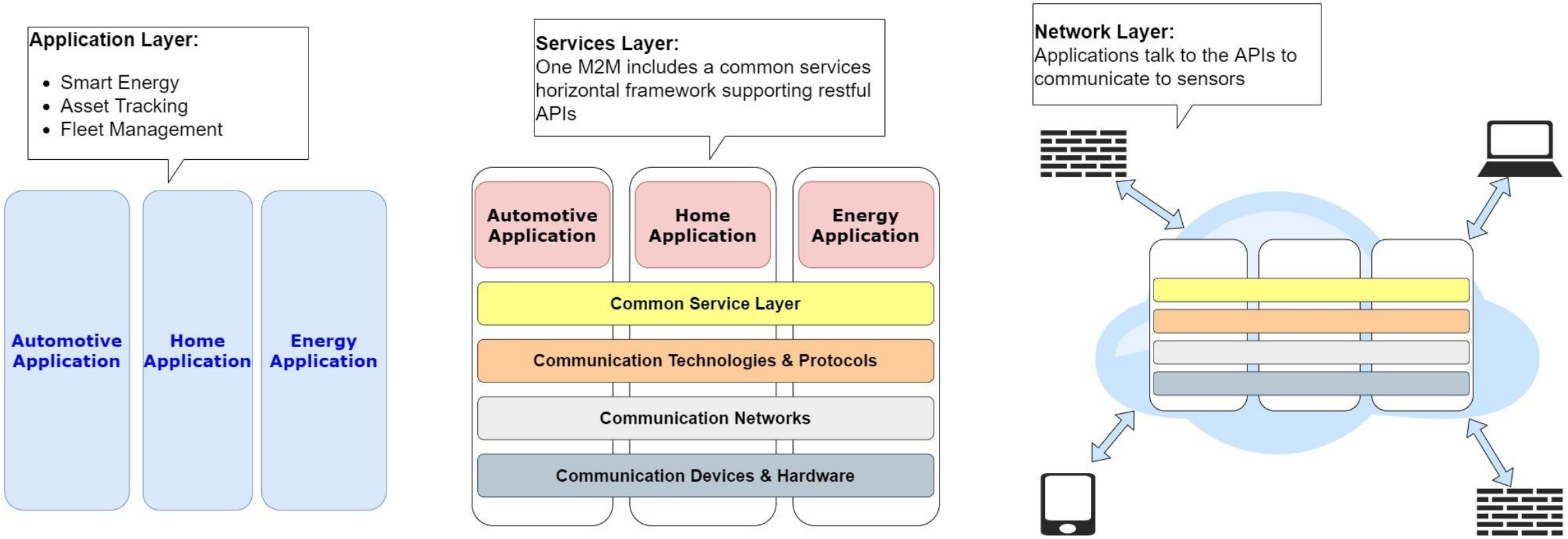
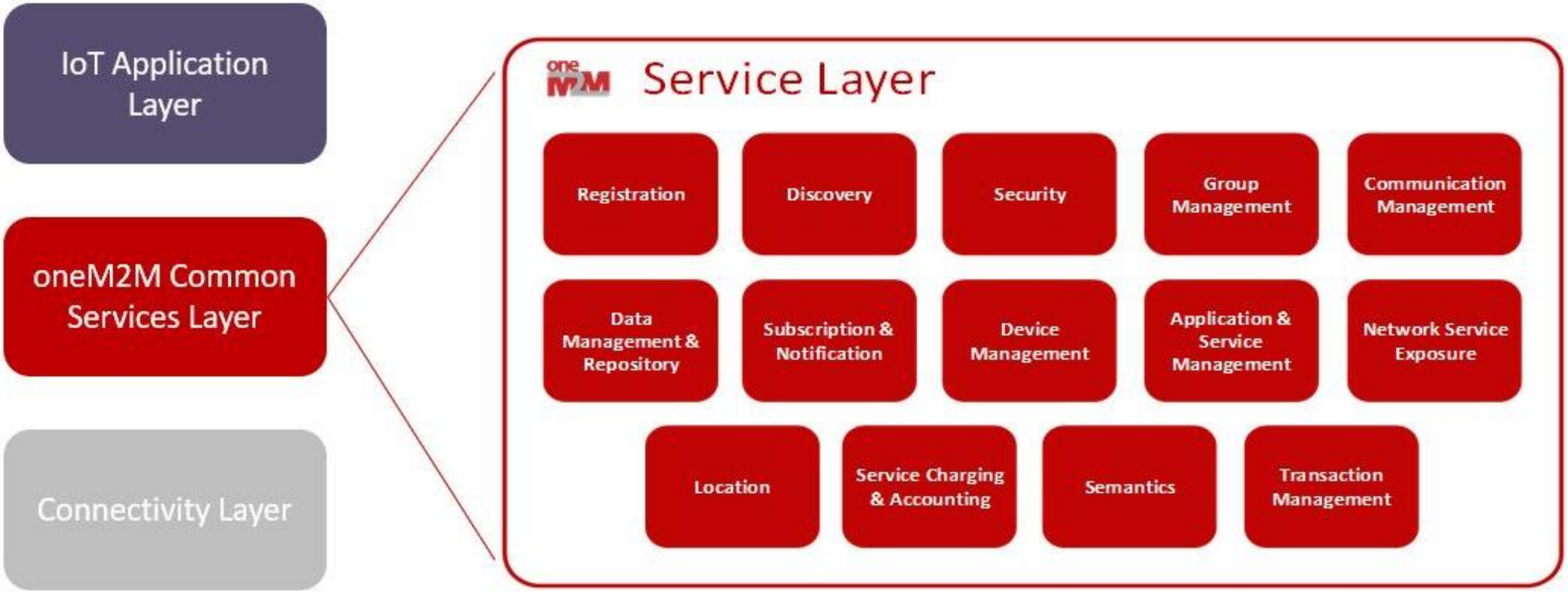


Fig: Main Elements of One M2M Architecture of IoT

Services Layer-One M2M



Vertical Silos vs. oneM2M Horizontal Platform

Vertical silos

Separate stack per application
Proprietary APIs
Device-to-app locked
High cost, slow roll-out

oneM2M horizontal

One common service layer
Standard RESTful APIs
Devices & data shared across apps
Lower cost, faster development

oneM2M Functional Architecture

Application Layer
AE – Application Entity

Common Services Layer
CSE – Common Services Entity

Network Services Layer
NSE – Network Services Entity

ADN
Application Dedicated Node – constrained device, AE only

ASN / MN
Application Service / Middle Node – device or gateway with CSE

IN
Infrastructure Node – cloud server with CSE

Reference Points

Mca
AE ↔ CSE: applications use the common services

Mcc
CSE ↔ CSE: gateway to cloud, peer-to-peer

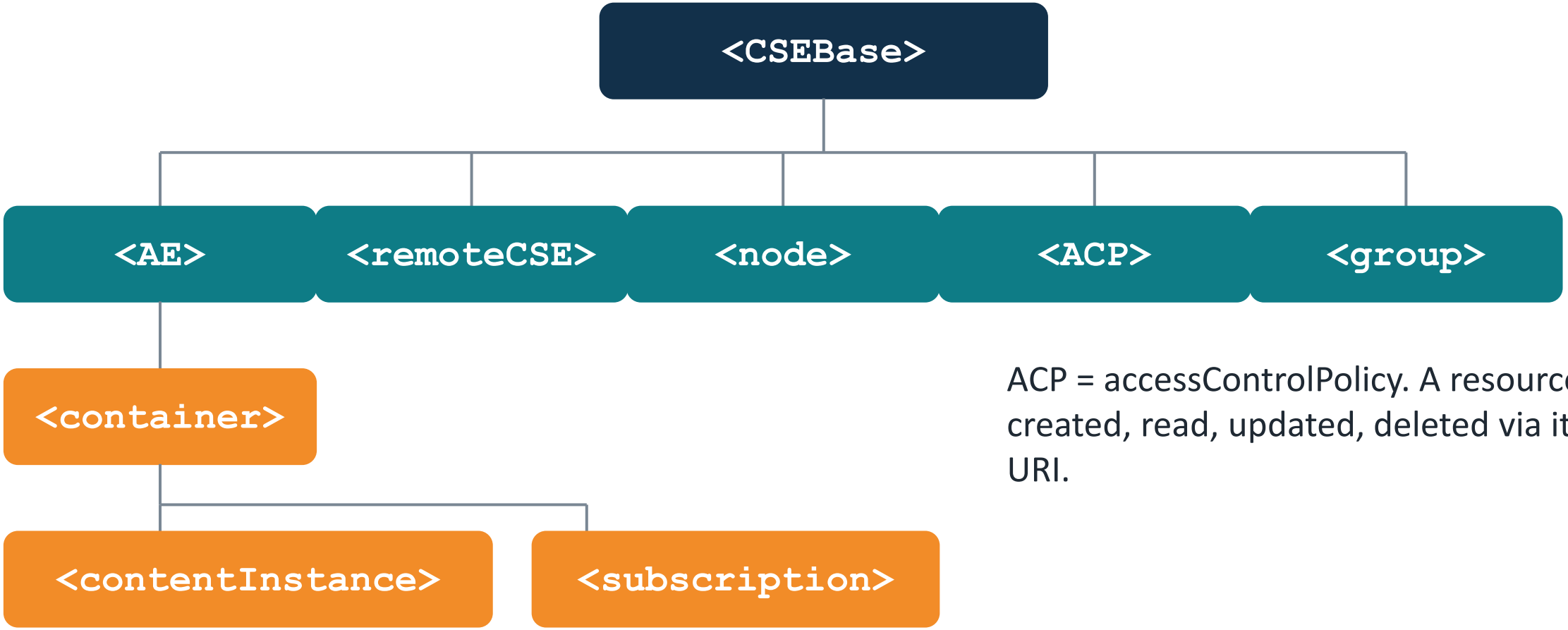
Mcn
CSE ↔ NSE: uses underlying network services

Mcc'
CSE ↔ CSE across different service providers

Common Services Functions (CSFs)

Registration	Discovery	Security
Group Management	Data Management & Repository	Subscription & Notification
Device Management	Location	Service Charging & Accounting

Resource Tree – Everything is a Resource



ACP = accessControlPolicy. A resource is created, read, updated, deleted via its URI.

RESTful Operations (CRUD + Notify)

Operation	Purpose	HTTP	CoAP
Create	Make a new resource	POST	POST
Retrieve	Read a resource	GET	GET
Update	Modify a resource	PUT	PUT
Delete	Remove a resource	DELETE	DELETE
Notify	Push change event	POST	POST

Protocol Bindings

HTTP
Request/response; widely supported on servers and gateways

CoAP
Lightweight over UDP; suited to constrained devices

MQTT
Publish/subscribe via broker; efficient for telemetry

WebSocket
Persistent two-way channel; low-latency notifications

Payload serialization: JSON or XML

Example: Temperature Sensor Flow

1. **Register AE**
   Sensor application registers with the CSE and receives an AE-ID

2. **Create <container>**
   Application creates a container named temperature

3. **Create <contentInstance>**
   Sensor posts each reading, e.g. 27.5 °C

4. **Create <subscription>**
   A dashboard app subscribes to the container

5. **Notify**
   CSE pushes new readings to the subscriber

Security and Interworking

Security

Access Control Policies (ACP)
TLS / DTLS transport security
Authentication & authorization
Secure registration of nodes

Interworking

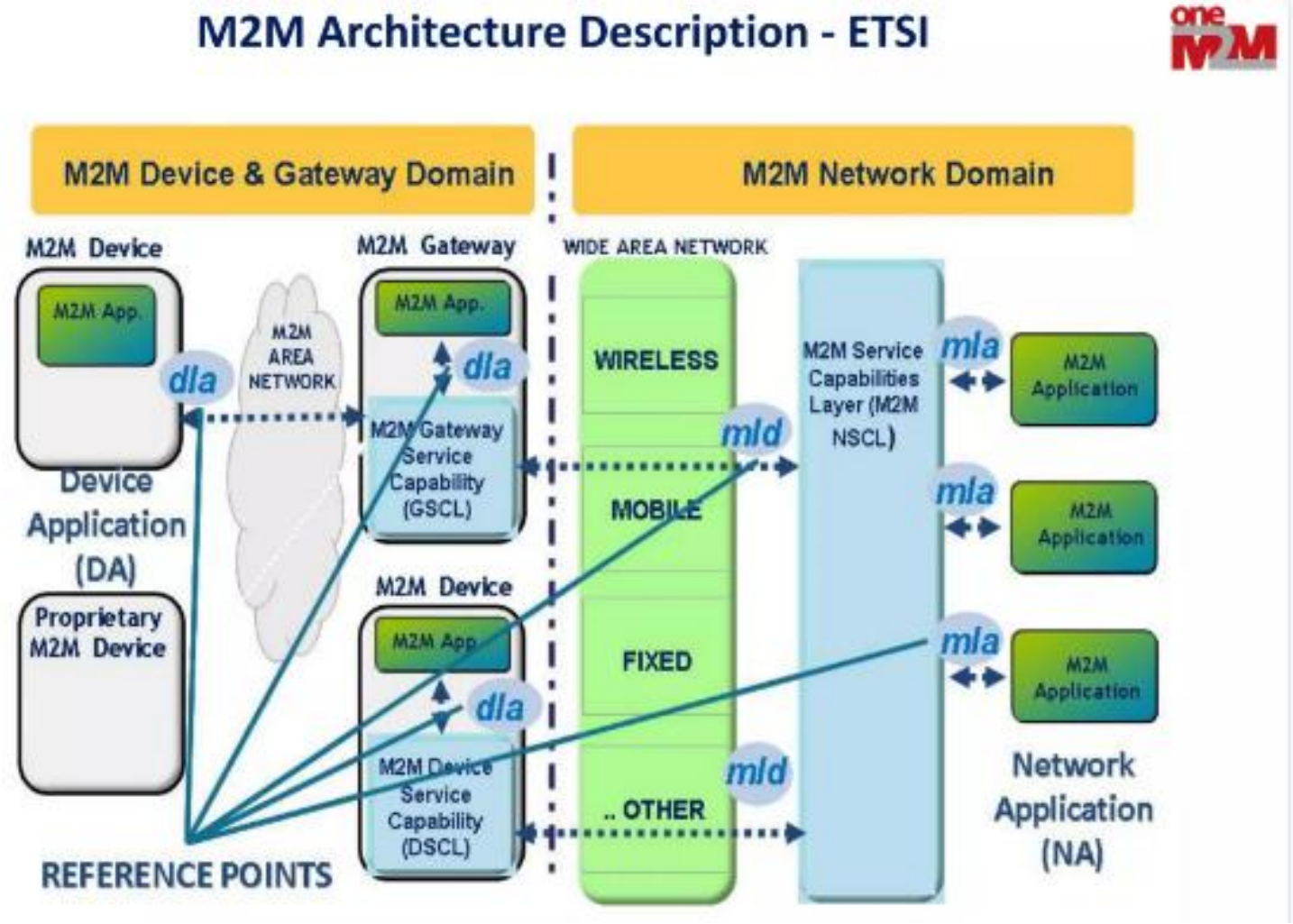
Interworking Proxy Entity (IPE)
Maps OMA LwM2M, Zigbee, BLE, OCF devices
Legacy devices appear as oneM2M resources
Device-to-cloud integration

Unit-5
Service Layer Protocols and Security

Services Layer-ETSI M2M

M2M Functional Architecture Overview

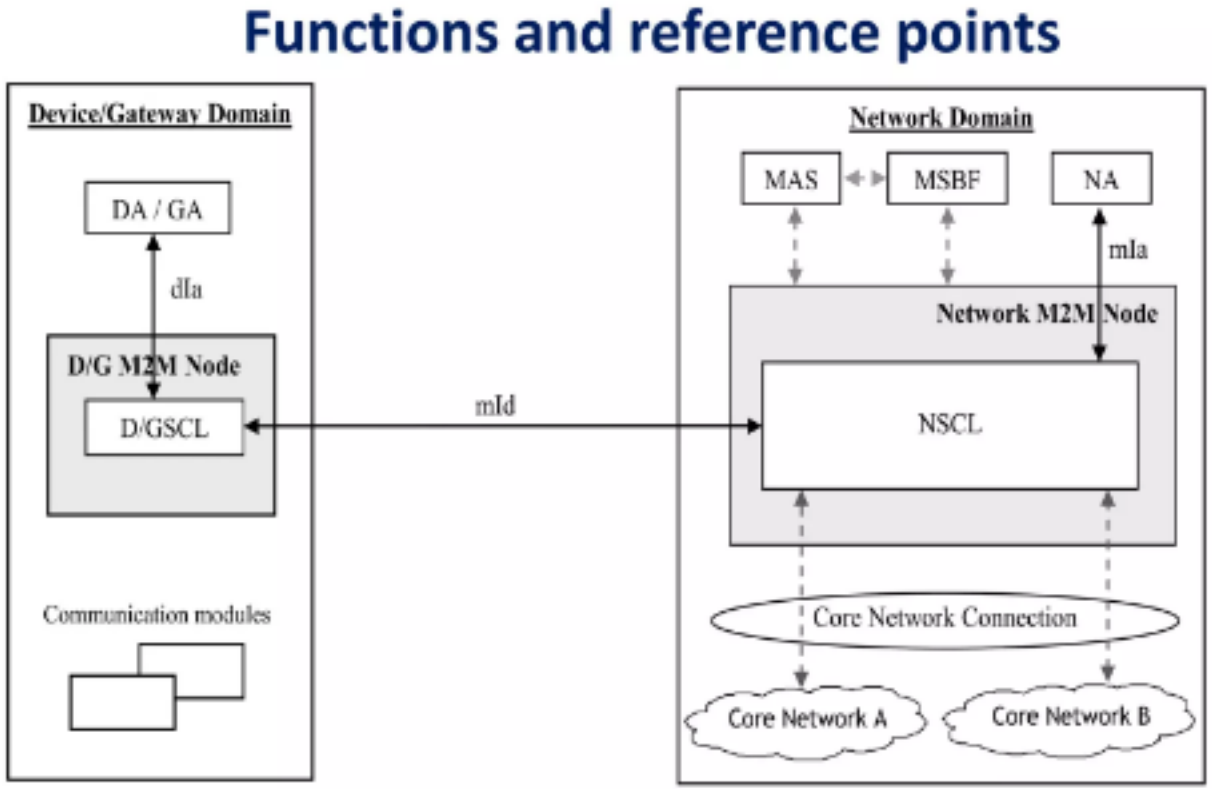
- mla: Message List Attribute
- mld: Message List Descriptor
- dla: Data List Attribute



European Telecommunications Standards Institute (ETSI)

Services Layer-ETSI M2M

M2M Architectural Framework



ETSI and the oneM2M Standard

- ETSI: European standards body; its TC M2M produced early M2M standards (2009 onward)
- Led to oneM2M (2012): global partnership of ETSI, TIA, ARIB, TTA, TTC, CCSA, ATIS, TSDSI
- Goal: one horizontal, technology-agnostic service layer for all verticals
- Releases 1 to 4+ add semantics, security, interworking and edge features
- Interworks with OCF, LwM2M, OPC-UA, 3GPP and others

oneM2M Layered Architecture

Application Layer

Application Entities (AEs): apps on devices, gateways and servers

Common Services Layer

Common Services Entity (CSE): registration, data management, security, subscription

Underlying Network Services Layer

Network Services Entity (NSE): connectivity, device triggering, location

- Mca: AE to CSE
- Mcc: CSE to CSE
- Mcn: CSE to NSE
- Mcc': CSE to CSE across service providers

Reference points

Functional Architecture: Nodes

Infrastructure Node

IN-CSE in the cloud; top of hierarchy; hosts IN-AEs

Middle Node

MN-CSE in a gateway; aggregates local devices

Application Service Node

ASN-CSE on a constrained device; ASN-AE

Application Dedicated Node

ADN: AE only, no CSE; very constrained sensors

Non-oneM2M Node

Legacy devices reached via interworking proxies

Common Services Functions (CSFs)

Registration (REG)

Register AEs and CSEs

Data Mgmt & Repository (DMR)

Store and share data

Discovery (DIS)

Find resources by attributes

Subscription & Notification (SUB)

Event-driven updates

Security (SEC)

Authentication, access control

Device Management (DMG)

Firmware, configuration

Resource Model and Protocol Bindings

- RESTful: everything is a resource with a URI in a tree rooted at CSEBase
- Operations: Create, Retrieve, Update, Delete, Notify (CRUD+N)
- Key resources: AE, container, contentInstance, subscription, group
- Data formats: JSON and XML

HTTP

Familiar web binding

CoAP

Constrained devices, UDP

MQTT / WebSocket

Pub-sub and persistent links

Key Takeaways-ETSI M2M

- ETSI M2M evolved into oneM2M: a horizontal IoT service layer
- Three layers: AE, CSE, NSE, connected by Mca, Mcc, Mcn
- Nodes: ADN, ASN, MN, IN form a flexible hierarchy
- CSFs give reusable registration, storage, discovery, security, subscriptions
- RESTful resources bound to HTTP, CoAP, MQTT, WebSocket

Discussion: How does a oneM2M gateway (MN-CSE) differ from a plain protocol gateway?

Recap: Where OMA Fits in the Service Layer



ETSI M2M

First horizontal M2M service layer; RESTful; basis for oneM2M



oneM2M

Global common service layer: CSE, AE, resource tree, interworking



OMA

Device management and service enablement for constrained devices

Today: OMA's role in the service layer, OMA DM, LwM2M architecture, object model, operations, security, and a comparison with oneM2M.

OMA at a Glance



Mission: interoperable service enablers, independent of bearer, OS and vendor



Founded 2002: born in mobile (SyncML, MMS), now focused on IoT



Since 2018: merged with IPSO Alliance as OMA SpecWorks



oneM2M link: oneM2M defines interworking with LwM2M devices

OMA specs for IoT

OMA DM

Device management over SyncML

LwM2M

Lightweight M2M for constrained nodes

NGSI

Context information interface

Why a Device-Management Layer?



Massive Scale

Millions of devices need remote configuration without site visits.



Constrained Nodes

Battery, kilobytes of RAM and lossy links demand light protocols.



Full Lifecycle

Bootstrap, monitor, update firmware and retire devices remotely.

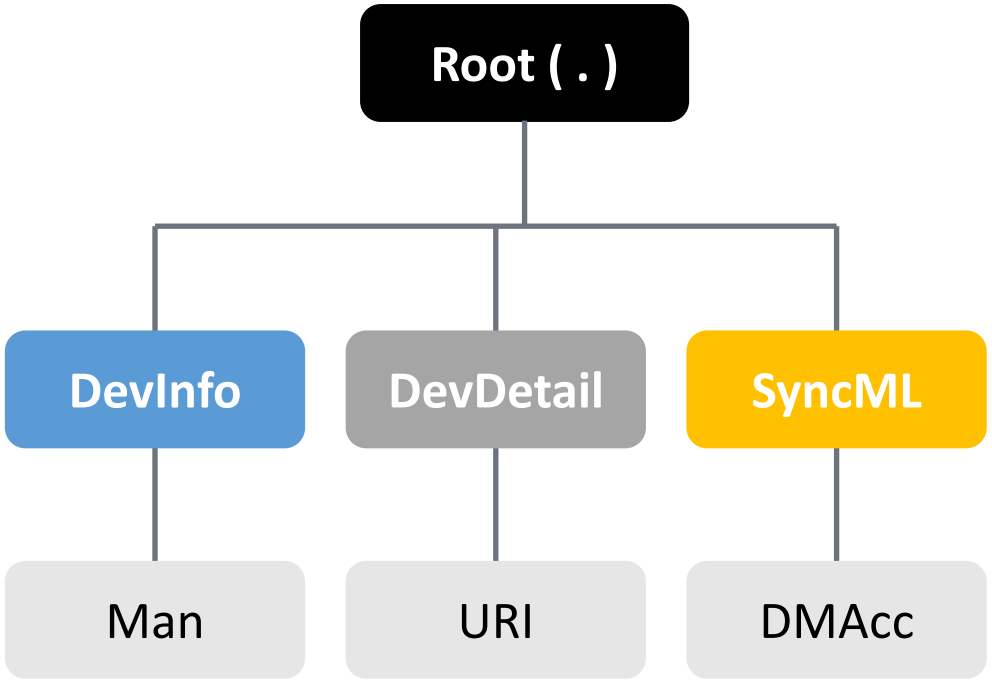


Security and Control

Credentials and access rights must be provisioned per server.

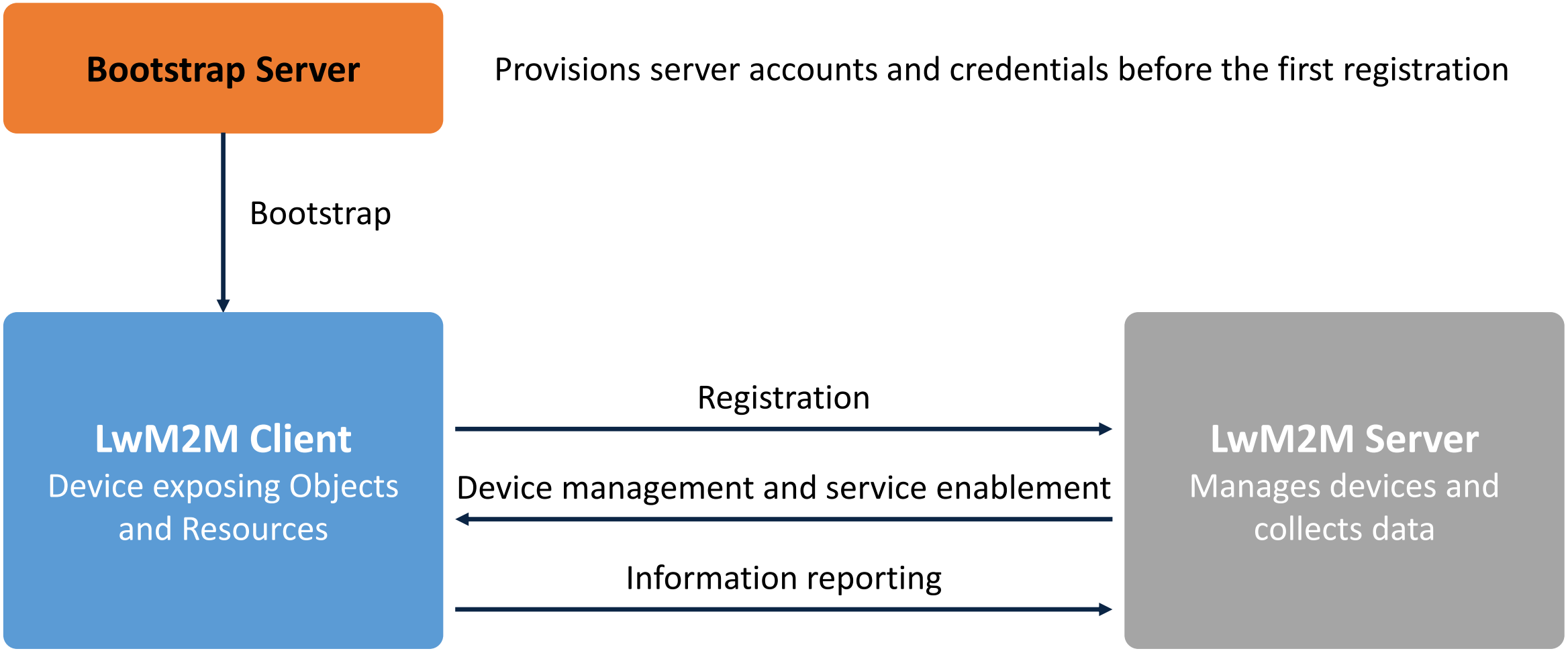
OMA DM: The Starting Point

- Device management protocol built on SyncML (XML)
- Management Tree: nodes addressed by URI
- Commands: Add, Get, Replace, Delete, Exec, Copy
- Server-initiated sessions via notification (e.g., SMS)
- Heavy XML payloads: unsuited to tiny sensors

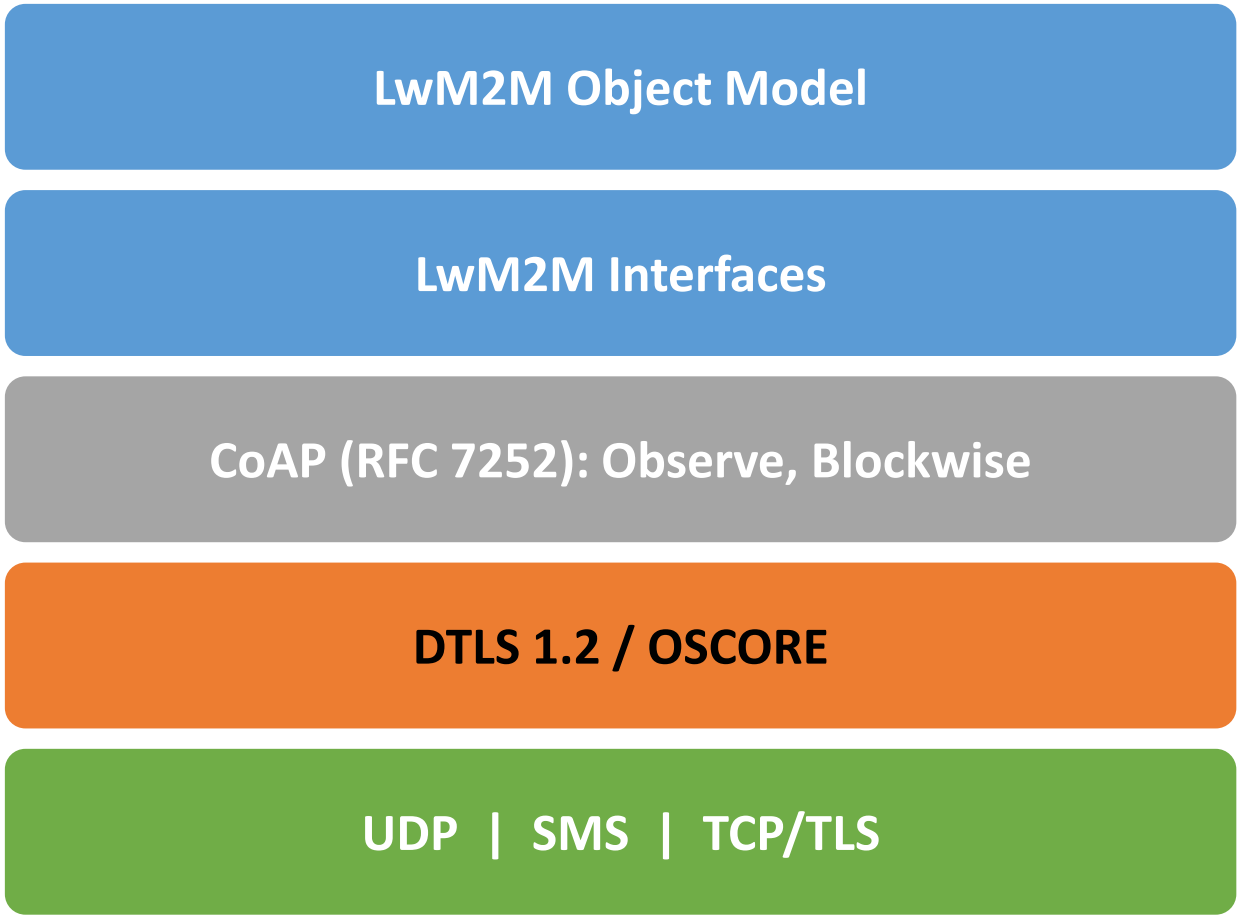


Example URI: ./DevInfo/Man

LwM2M Architecture



LwM2M Protocol Stack



Default binding: UDP; SMS for cellular wake-up

Payload formats: TLV, JSON; SenML CBOR/JSON from v1.1

Footprint: fits Class-1 constrained devices (about 10 KB RAM)

LwM2M Object Model

/3/0/0

3	Object ID: Device object
0	Instance ID: first instance
0	Resource ID: Manufacturer

IPSO example: /3303/0/5700 = temperature sensor value

ID	Standard Object
0	Security
1	Server
2	Access Control
3	Device
4	Connectivity Monitoring
5	Firmware Update
6	Location
7	Connectivity Statistics

Operations on Resources

Operation	CoAP Method	Purpose
Read	GET	Retrieve a resource value
Write	PUT / POST	Change a resource value
Execute	POST	Trigger an action, e.g., reboot
Create	POST	Create an object instance
Delete	DELETE	Remove an object instance
Discover	GET (link-format)	List objects and resources
Write-Attributes	PUT	Set notification rules (pmin, pmax)
Observe / Notify	GET + Observe	Push value changes to the server

Device Lifecycle: Bootstrap to Operate

1

Bootstrap

Client receives server address and credentials (factory, client- or server-initiated)

2

Register

Client sends endpoint name, lifetime and supported objects to the server

3

Operate

Server reads, writes, executes; client reports via Observe and Notify

4

Maintain

Periodic Update before lifetime expires; Deregister on shutdown

Registration carries: endpoint name, lifetime, LwM2M version, binding mode, object list

LwM2M Version Evolution



2017

- CoAP over UDP and SMS
- DTLS security
- TLV and JSON payloads
- Four core interfaces

2018

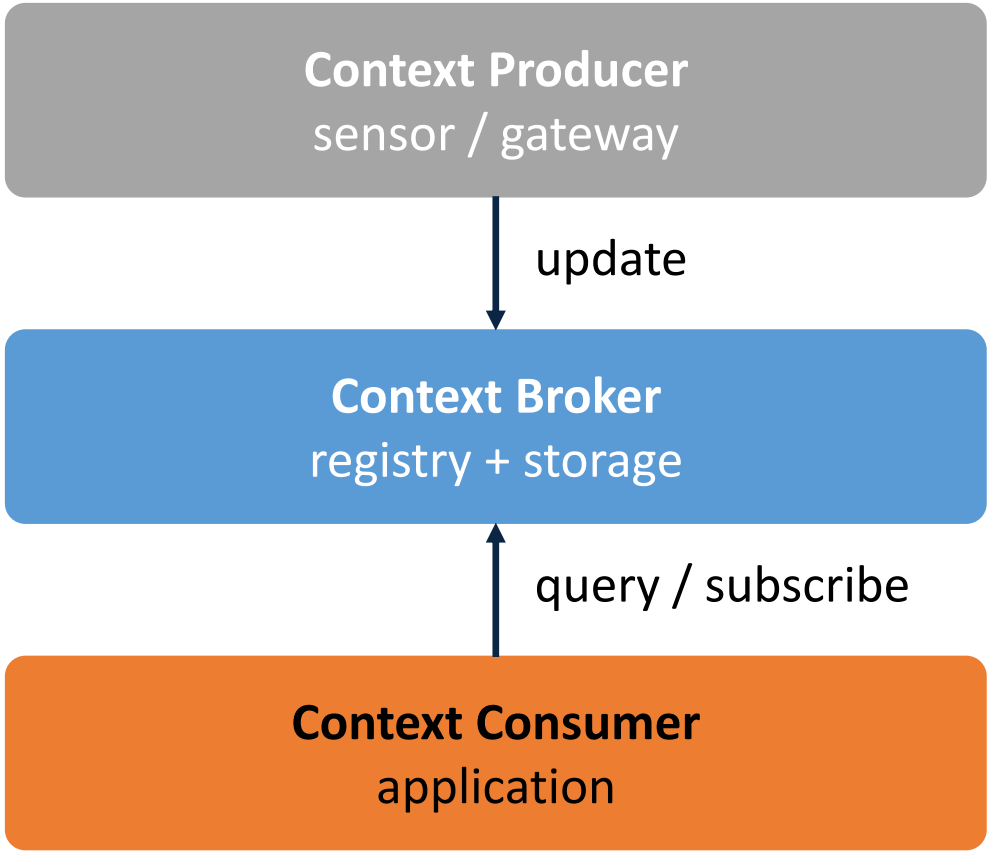
- TCP/TLS binding
- SenML CBOR/JSON
- Non-IP (NIDD), X.509
- Send, composite operations

2020

- MQTT and HTTP bindings
- OSCORE security
- Gateway functionality

OMA NGSI: Context Information

- NGSI: Next Generation Service Interface
- Entities carry attributes and metadata
- NGSI-9: register and discover context providers
- NGSI-10: query, update and subscribe
- Evolved into ETSI NGSI-LD (JSON-LD); FIWARE brokers implement it



Security in LwM2M



Transport Security

DTLS 1.2 with PSK, raw public keys or X.509; OSCORE added in v1.2



Secure Bootstrap

Factory, client- or server-initiated bootstrap provisions keys safely



Access Control

Access Control object (ID 2) grants each server rights per instance

Security object (ID 0) stores keys; Server object (ID 1) stores lifetime and binding

LwM2M vs oneM2M vs ETSI M2M

Aspect	ETSI M2M	oneM2M	OMA LwM2M
Focus	Horizontal M2M service layer	Common IoT service layer	Device management
Data model	Resource tree (RESTful)	Resource tree (RESTful)	Objects and Resources
Protocols	HTTP, CoAP	HTTP, CoAP, MQTT, WebSocket	CoAP/UDP, SMS; MQTT, HTTP in v1.2
Target	Gateways and M2M apps	Cloud, gateways, apps	Constrained devices
Status	Evolved into oneM2M	Active, global	Active (OMA SpecWorks)

oneM2M TS-0014 defines interworking with LwM2M devices via an IPE

Key Takeaways -OMA

- OMA complements oneM2M and ETSI M2M: it manages and enables devices
- LwM2M = CoAP + object model + four interfaces
- Lifecycle: Bootstrap, Register, Operate, Maintain
- oneM2M can interwork with LwM2M devices (TS-0014)

Think about it

1. Why is CoAP preferred over HTTP for LwM2M clients?

2. How would an IPE expose /3303/0/5700 in oneM2M?

Recap: From ETSI M2M to oneM2M to BBF

ETSI M2M

Device, gateway and network domains
Service Capability Layers (SCL)
RESTful resources

oneM2M

Common Services Functions (CSFs)
CSE and AE entities
Global horizontal platform

BBF

Device and service management
Home gateways, CPE, IoT
TR-069 to TR-369 (USP)

Common goal: interoperable, standards-based machine communication. BBF's angle: managing the devices at the network edge.

Broadband Forum (BBF) at a glance

1994

DSL Forum founded

2008

Renamed Broadband Forum

- Global consortium of service providers, equipment vendors and chipmakers
- Scope: access networks, home networks, device management, cloud
- Output: Technical Reports (TR-xxx), published as open specifications
- Best known in IoT for TR-069 (CWMP) and TR-369 (USP)

Why BBF matters for IoT



- Operators already manage millions of gateways remotely
- The same management channel can be extended to IoT services
- Remote configuration, monitoring, upgrade and troubleshooting
- Standard data models avoid vendor lock-in

BBF specification family

TR-069

CPE WAN Management Protocol (CWMP)

TR-098

Internet Gateway Device data model (legacy)

TR-106

Rules for writing data models

TR-157

Component objects for CWMP

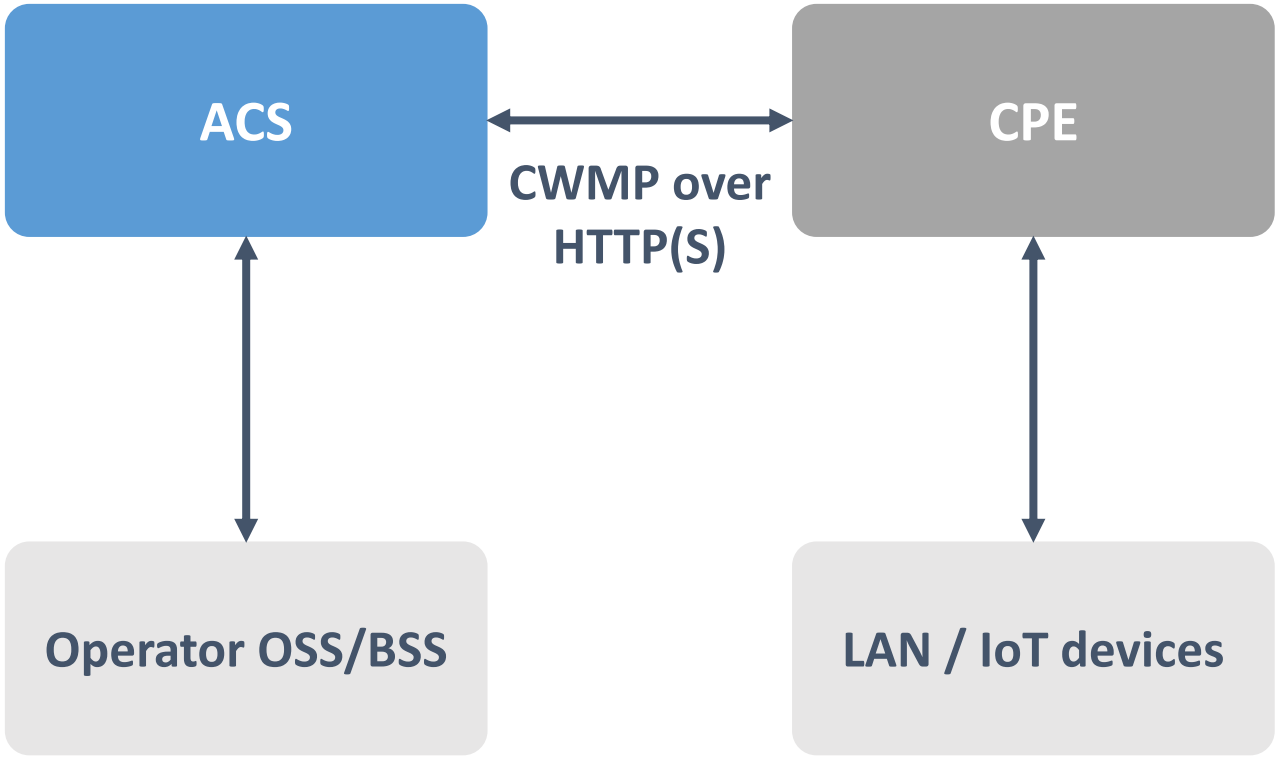
TR-181

Device:2 data model

TR-369

User Services Platform (USP)

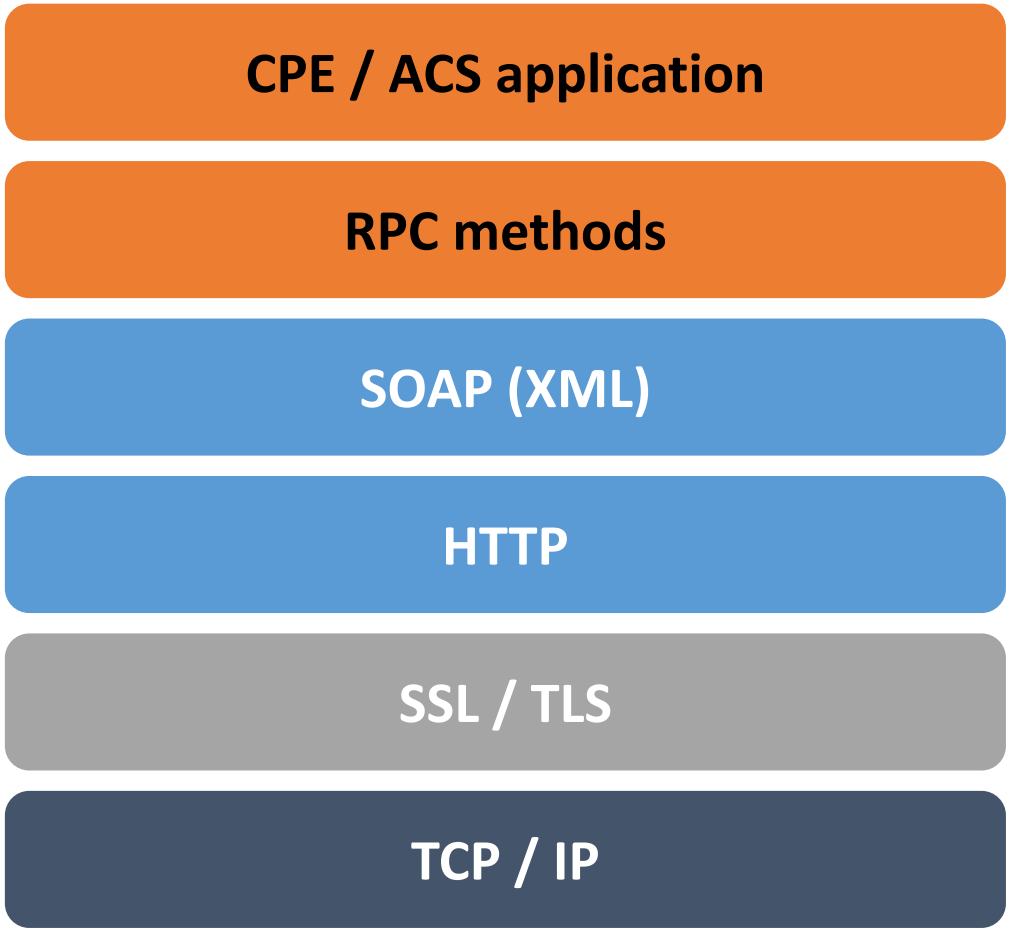
TR-069 (CWMP): Architecture



ACS = Auto-Configuration Server

- CPE is the client; ACS is the server
- CPE always initiates the session, so NAT and firewalls are not a problem
- Connection Request lets the ACS prompt a session
- First published in 2004 for DSL customer premises equipment

CWMP protocol stack



- RPC methods carry the management intent
- SOAP/XML encodes every message
- HTTP is the carrier; TLS secures it
- Authentication: HTTP Basic or Digest, plus certificates

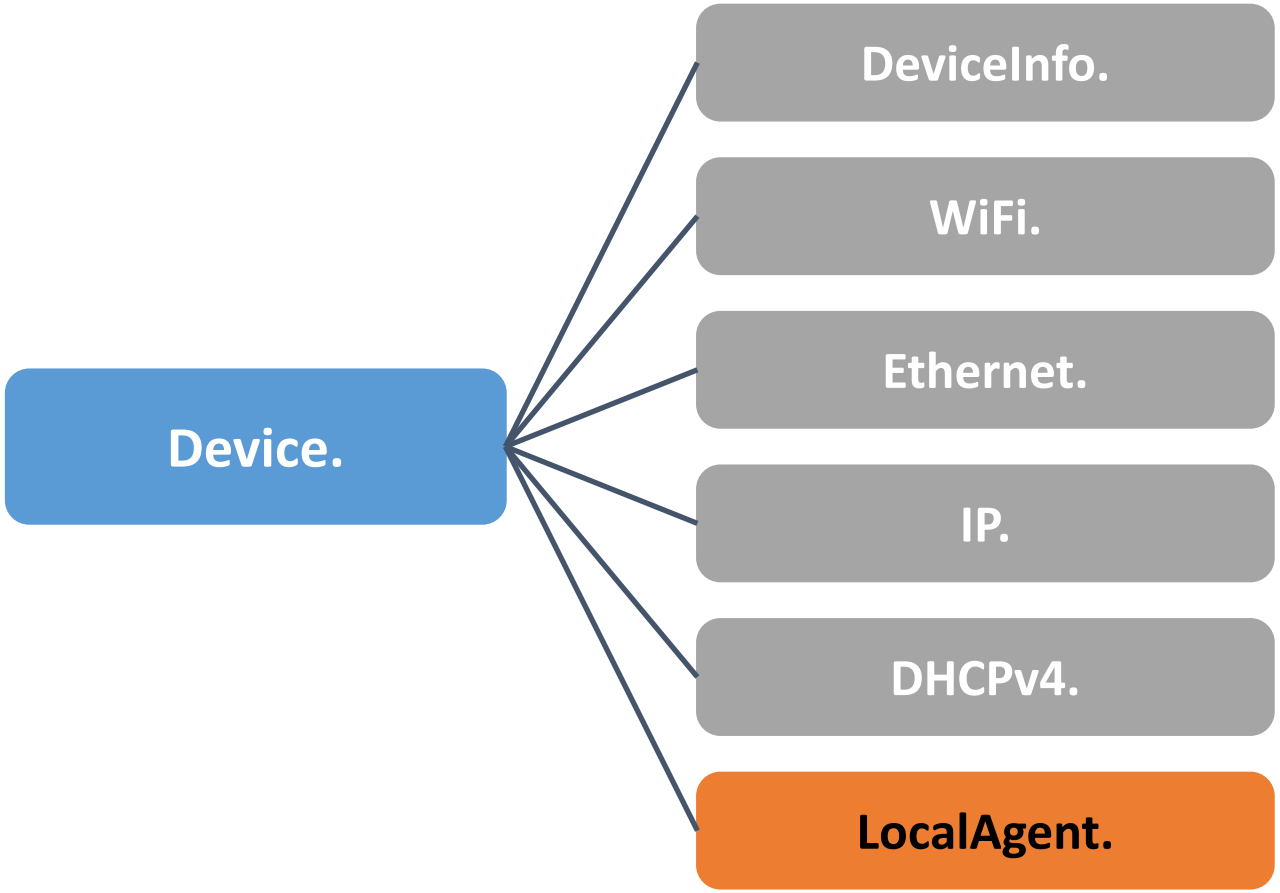
CWMP RPC methods

Method	Purpose
GetRPCMethods	Discover supported methods
Get / SetParameterValues	Read and write data model parameters
GetParameterNames / Attributes	Browse the model, manage notifications
AddObject / DeleteObject	Create or remove object instances
Download / Upload	Firmware, configuration and log files
Reboot / FactoryReset	Device lifecycle control
Inform (CPE to ACS)	Start a session, report events

Anatomy of a CWMP session

1. **CPE to ACS:** Inform, with events such as BOOT, PERIODIC, VALUE CHANGE
2. **ACS to CPE:** InformResponse
3. **CPE to ACS:** Empty HTTP POST: ready for requests
4. **ACS to CPE:** RPC request, e.g. SetParameterValues
5. **CPE to ACS:** RPC response, repeated for further requests
6. **ACS to CPE:** Empty response closes the session

TR-181: The Device:2 data model



- Hierarchical tree of objects and parameters
- Example path: Device.WiFi.SSID.1.SSID
- Numbered instances: .1., .2., ...
- Shared by CWMP and USP
- LocalAgent. holds the USP Agent settings

Why move beyond TR-069?

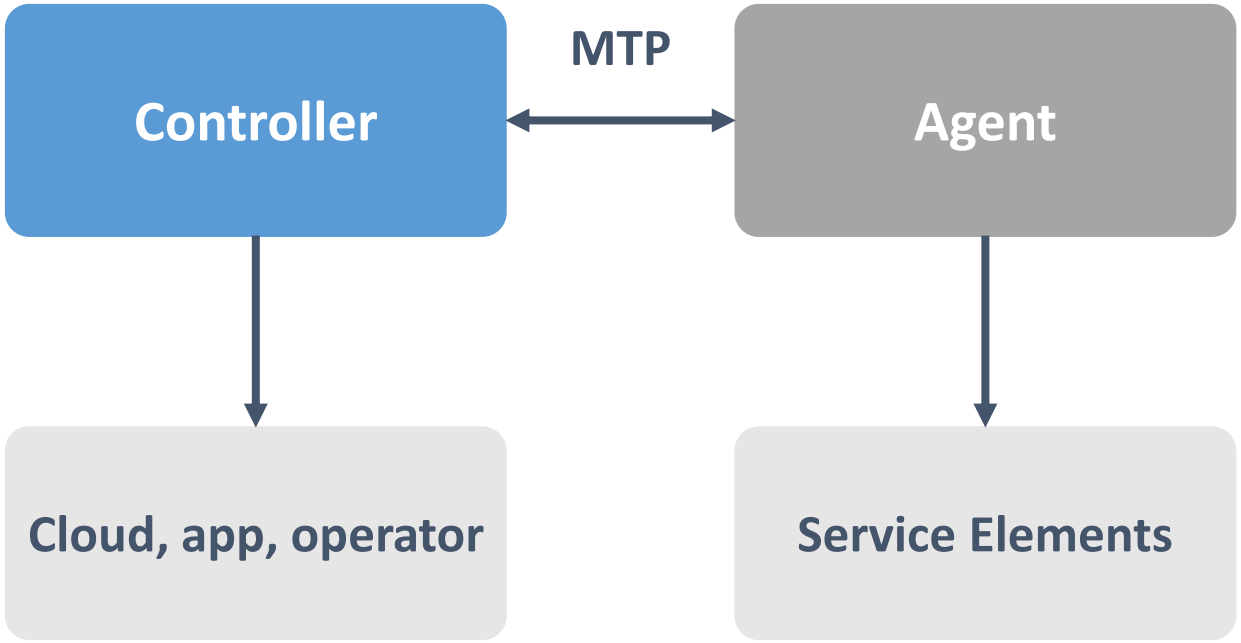
TR-069 limitations

- Single controller: one ACS per CPE
- CPE polling; no native push path
- Heavy SOAP/XML for small devices
- Focused on the gateway, not apps

What IoT needs

- Multiple controllers: operator, user, apps
- Real-time notifications
- Lightweight, extensible encoding
- Reach devices behind the gateway

TR-369 (USP): Architecture



- Agent: runs on the device, exposes Service Elements
- Controller: manages one or more Agents
- One Agent can serve several Controllers
- Defined by BBF as TR-369

Each side is an Endpoint with a unique Endpoint ID

USP message types

Add

Create new object instances

Set

Update parameter values

Get

Read parameters and objects

Delete

Remove object instances

Operate

Run commands such as reboot or upgrade

Notify

Events and value-change reports

Also: GetInstances, GetSupportedDM, GetSupportedProtocol. Messages are encoded with Protocol Buffers.

USP message transfer protocols

CoAP

Lightweight, UDP-based; suits constrained devices

STOMP

Broker-based messaging with queues

WebSocket

Persistent, bidirectional channel over HTTP/TLS

MQTT

Publish/subscribe through a broker

Encoding (protobuf) is independent of the MTP: the same USP record travels over any transport, with end-to-end security.

CWMP vs USP

Feature	TR-069 (CWMP)	TR-369 (USP)
Topology	One ACS per CPE	Many Controllers and Agents
Session start	CPE-initiated	Either side; asynchronous
Transport	HTTP / SOAP	CoAP, STOMP, WebSocket, MQTT
Encoding	XML	Protocol Buffers (binary)
Security	TLS (transport)	TLS + end-to-end + roles
Data model	TR-098 / TR-181	TR-181 (Device:2)

ETSI M2M vs oneM2M vs BBF

	ETSI M2M	oneM2M	BBF (USP)
Focus	M2M service capabilities	Horizontal IoT platform	Device and service management
Entities	DSCL, GSCL, NSCL	AE, CSE	Controller, Agent
Model	RESTful resources	RESTful resources	TR-181 objects
Bindings	HTTP, CoAP	HTTP, CoAP, MQTT, WebSocket	CoAP, STOMP, WebSocket, MQTT
Typical use	Early M2M deployments	Cross-domain interoperability	Gateways, CPE, smart home

Where BBF protocols are used

Zero-touch provisioning

Gateways self-configure and upgrade firmware on first connection

Smart-home control

A mobile app acts as a USP Controller for the home gateway

Wi-Fi and mesh management

Monitor and tune access points across the home network

Remote troubleshooting

Diagnostics and notifications reach the operator in real time

Key takeaways-BBF

1. BBF standardises remote management of broadband devices, now extended to IoT.
2. TR-069 (CWMP): ACS and CPE, SOAP over HTTP, CPE-initiated sessions.
3. TR-181 is the common data model used by both protocols.
4. TR-369 (USP): multiple controllers, event-driven, several MTPs, protobuf.
5. BBF complements ETSI M2M and oneM2M: device layer versus service platform.

Unit-5
Service Layer Protocols and Security

Why Security in IoT?

IoT devices and networks are vulnerable to various cyber threats



Unit-5
Service Layer Protocols and Security

Why Security in IoT?



- **Data Breaches**: Personal, financial, or health data can be intercepted or stolen.
- **Privacy Risks**: Unauthorized access can expose private information.
- **Device Hijacking**: Attackers can control devices for malicious purposes, such as spying or disruption.
- **Botnet Attacks**: Insecure devices can be used in large-scale attacks like DDoS.
- **Physical Damage**: Compromised IoT systems controlling infrastructure could cause physical harm or system failures.
- **Regulatory Compliance**: Lack of security can lead to violations of privacy laws and regulations.

Unit-5
Service Layer Protocols and Security

Security in IoT Protocols

- Security in **IoT protocols** is a critical concern due to the vast number of connected devices, the diverse nature of IoT systems, and the sensitive data they handle.
- The primary objective of security in IoT is
- to ensure the **confidentiality**, **integrity**, and **availability** of data and devices,
- as well as to provide **authentication**, **authorization**, and **non-repudiation**.

Unit-5
Service Layer Protocols and Security

Security in IoT Protocols

Key Security Concerns in IoT

1.Device Authentication: Ensuring that devices can authenticate themselves to the network or to other devices to prevent unauthorized access.

2.Data Encryption: Ensuring that data is encrypted both in transit and at rest to prevent interception and tampering.

3.Access Control: Ensuring that only authorized users and devices can access certain functions or data.

Unit-5
Service Layer Protocols and Security

Security in IoT Protocols

Key Security Concerns in IoT

4. **Integrity and Non-repudiation**: Ensuring that data cannot be altered without detection, and that actions can be attributed to specific actors or devices.
5. **Network Security**: Protecting the communication channels used by IoT devices from attacks such as eavesdropping, man-in-the-middle, and denial of service.
6. **Device Security**: Protecting the physical devices themselves from being compromised or tampered with.

Unit-5
Service Layer Protocols and Security

Security in IoT Protocols

IoT protocols facilitate communication, control, and data exchange among devices. However, due to their limited resources and lack of robust security, IoT protocols are vulnerable to various attacks. Below are the attacks categorized by protocol type:

1. Attacks on IEEE 802.15.4 (MAC/PHY Layer)

Eavesdropping: Interception of unencrypted wireless communication.

Jamming: Disrupting communication by broadcasting interference.

Replay Attack: Resending previously captured frames.

Beacon Flooding: Overloading the network with fake beacon frames.

Collision Attack: Deliberate interference causing packet loss.

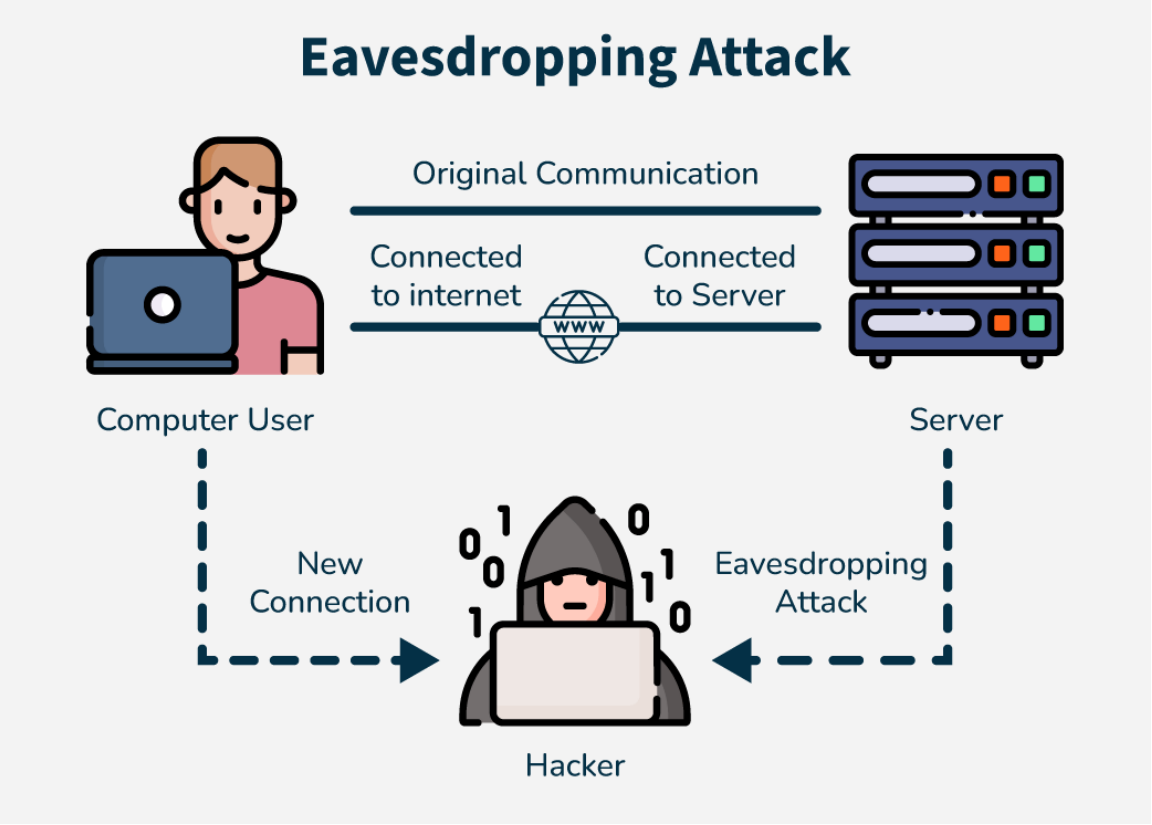
Spoofing: Forging device identities (MAC address).

Unit-5
Service Layer Protocols and Security

Security in IoT Protocols

1. Attacks on IEEE 802.15.4 (MAC/PHY Layer)

Eavesdropping: Interception of unencrypted wireless communication.

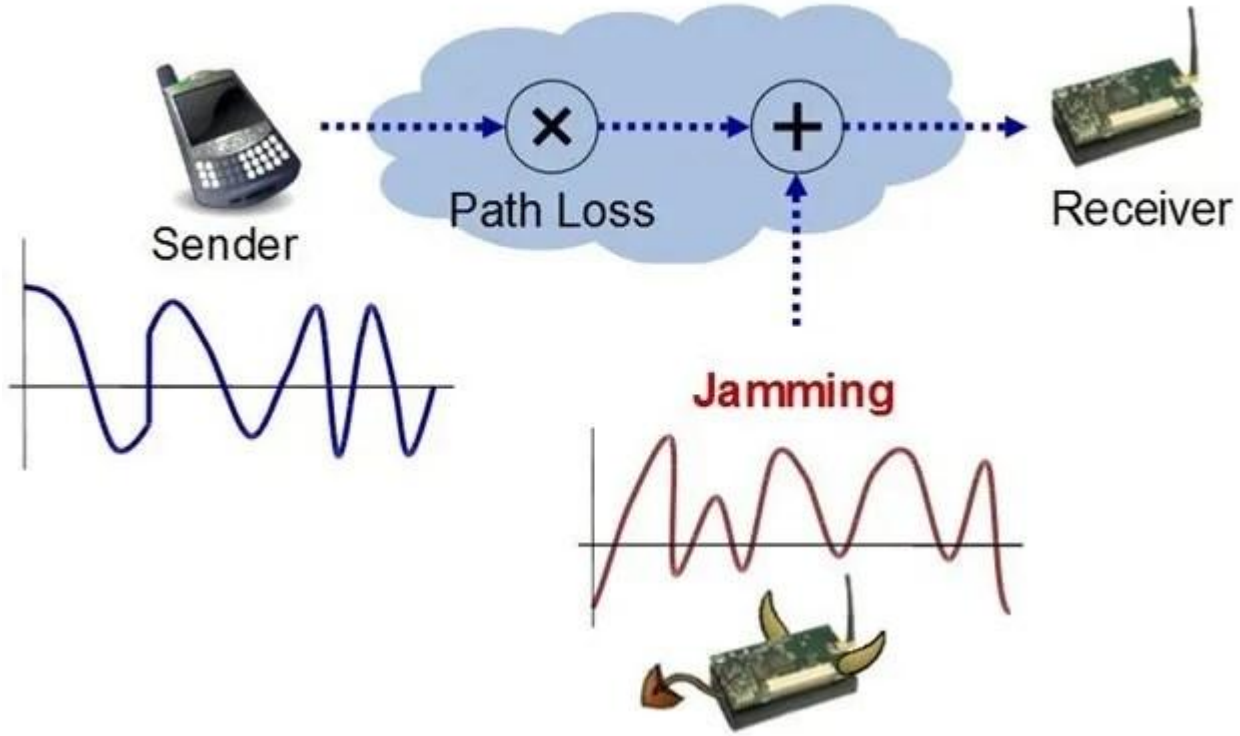


Unit-5
Service Layer Protocols and Security

Security in IoT Protocols

1. Attacks on IEEE 802.15.4 (MAC/PHY Layer)

Jamming: Disrupting communication by broadcasting interference.

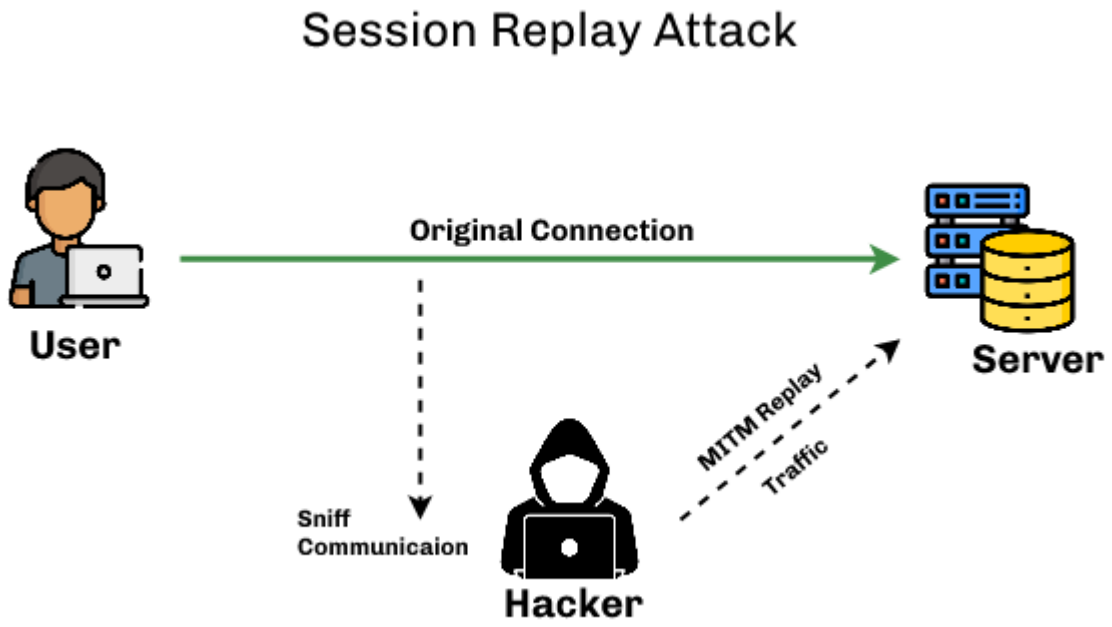


Unit-5
Service Layer Protocols and Security

Security in IoT Protocols

1. Attacks on IEEE 802.15.4 (MAC/PHY Layer)

Replay Attack: Resending previously captured frames.

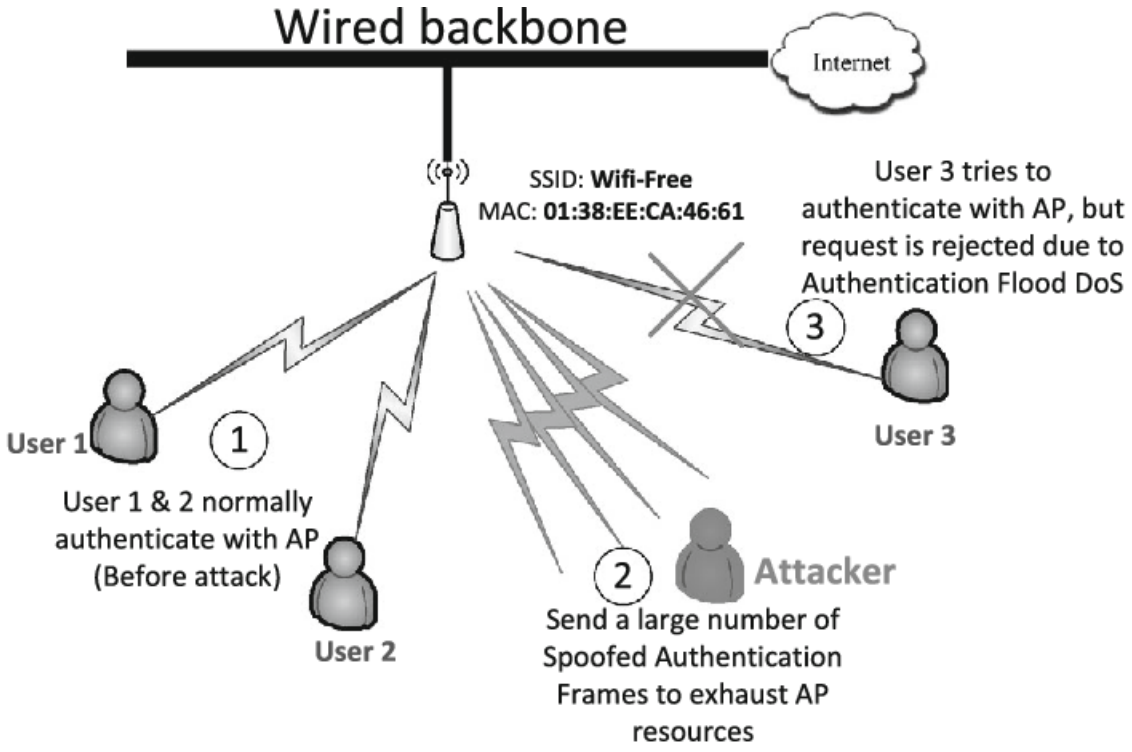


Unit-5
Service Layer Protocols and Security

Security in IoT Protocols

1. Attacks on IEEE 802.15.4 (MAC/PHY Layer)

Beacon Flooding: Overloading the network with fake beacon frames.



Unit-5
Service Layer Protocols and Security

Security in IoT Protocols

1. Attacks on IEEE 802.15.4 (MAC/PHY Layer)

Collision Attack: Deliberate interference causing packet loss.

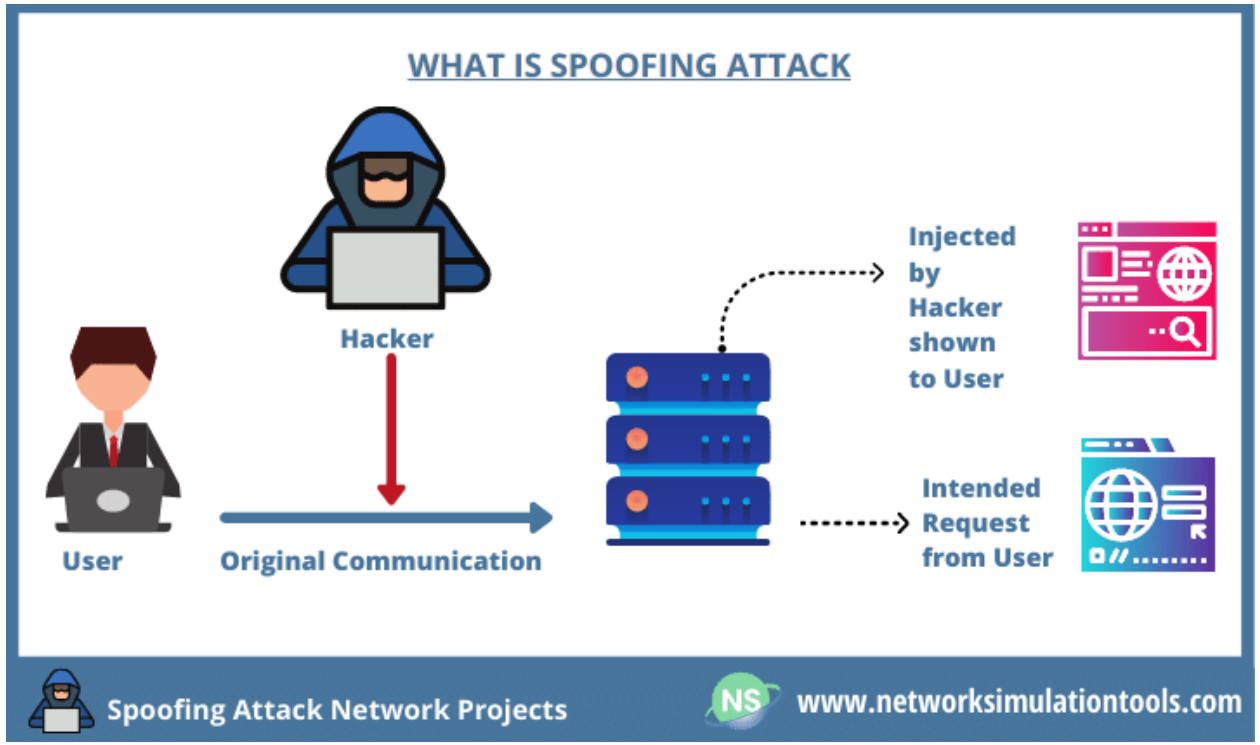


Unit-5
Service Layer Protocols and Security

Security in IoT Protocols

1. Attacks on IEEE 802.15.4 (MAC/PHY Layer)

Spoofing: Forging device identities (MAC address).



Unit-5
Service Layer Protocols and Security

Security in IoT Protocols

2. Attacks on Zigbee

Key Extraction: Exploiting weak key management to retrieve encryption keys.

Device Cloning: Duplicating legitimate devices to gain unauthorized access.

Over-the-Air Update Exploitation: Tampering with firmware updates.

Denial of Service (DoS): Flooding or jamming the Zigbee network.

Unit-5
Service Layer Protocols and Security

Security in IoT Protocols

3. Attacks on MQTT (Message Queuing Telemetry Transport)

Eavesdropping: Capturing messages due to lack of encryption.

Man-in-the-Middle (MITM): Intercepting and modifying messages between clients and brokers.

Replay Attack: Resending previously captured MQTT messages.

Authentication Bypass: Exploiting weak or missing authentication mechanisms.

Payload Injection: Sending malicious payloads to manipulate device behavior.

Unit-5
Service Layer Protocols and Security

Security in IoT Protocols

4. Attacks on CoAP (Constrained Application Protocol)

Eavesdropping: Intercepting unencrypted CoAP messages.

Replay Attack: Resending legitimate requests to exhaust device resources.

Message Forgery: Injecting fake CoAP messages to manipulate device responses.

Amplification Attack: Using CoAP's large responses to launch reflection attacks.

DoS Attack: Flooding CoAP servers with excessive requests.

Unit-5
Service Layer Protocols and Security

Security in IoT Protocols

5. Attacks on 6LoWPAN

Routing Attacks: Exploiting vulnerabilities in routing protocols (e.g., RPL).

Wormhole Attack: Tunneling packets between two colluding nodes.

Black Hole Attack: Dropping all received packets.

Selective Forwarding: Dropping selected packets.

Header Manipulation: Tampering with compressed headers to disrupt communication.

Energy Depletion: Exploiting routing or communication to drain battery.

Unit-5
Service Layer Protocols and Security

Security in IoT Protocols

6. Attacks on Bluetooth Low Energy (BLE)

Sniffing: Intercepting BLE communication during pairing or data exchange.

Impersonation: Exploiting weak authentication to act as a trusted device.

Replay Attack: Resending captured BLE messages.

DoS: Overloading BLE devices with connection requests.

Unit-5
Service Layer Protocols and Security

Security in IoT Protocols

7. Attacks on LoRaWAN

Eavesdropping: Capturing unencrypted or weakly encrypted LoRaWAN packets.

Key Extraction: Exploiting key-sharing mechanisms to retrieve encryption keys.

Frame Replay: Resending valid frames to cause disruptions.

Gateways Overload: Flooding LoRa gateways with excessive traffic.

8. Attacks on HTTP/HTTPS

MITM: Intercepting communication by exploiting weak TLS configurations

Phishing: Redirecting IoT devices to malicious web servers.

DoS/DDoS: Overloading IoT web servers with excessive requests.

Unit-5
Service Layer Protocols and Security

Security in IoT Protocols

9. Attacks on WebSocket

Cross-Site WebSocket Hijacking: Exploiting browser vulnerabilities to intercept WebSocket traffic.

Payload Injection: Sending malicious data through WebSocket connections.

Resource Exhaustion: Overloading the WebSocket server with connections.

Unit-5
Service Layer Protocols and Security

Security in IoT Protocols

10. Attacks on AMQP (Advanced Message Queuing Protocol)

Message Injection: Sending unauthorized messages to IoT queues.

MITM: Intercepting and altering AMQP messages.

Queue Flooding: Overloading the broker with excessive messages.

Unit-5
Service Layer Protocols and Security

Security in IoT Protocols

11. General Protocol-Level Attacks

Brute Force: Attempting to guess credentials for protocol authentication.

Downgrade Attacks: Forcing devices to use weaker security versions of protocols.

Protocol Exploitation: Exploiting unmatched vulnerabilities in protocol implementations.

Unit-5
Service Layer Protocols and Security

Security in IoT Protocols

Mitigation Strategies

Encryption: Use end-to-end encryption (e.g., TLS, DTLS).

Authentication: Implement robust authentication mechanisms, such as certificates or tokens.

Replay Protection: Use nonces or timestamps to prevent replay attacks.

Secure Key Management: Ensure keys are stored securely and exchanged using secure protocols.

Regular Updates: Patch vulnerabilities in protocol implementations.

Traffic Monitoring: Deploy intrusion detection systems (IDS) to identify anomalous traffic.

Unit-5
Service Layer Protocols and Security

Security in IEEE 802.15.4

The IEEE 802.15.4 standard, which is commonly used for low-rate wireless personal area networks (LR-WPANs), includes several security features to ensure secure communication.

1. **Encryption**: The standard uses the Advanced Encryption Standard (AES) with a 128-bit key length to encrypt data. This ensures that data transmitted over the network is protected from eavesdropping.
2. **Message Integrity Code (MIC)**: To ensure data integrity and authenticity, a MIC (also known as a Message Authentication Code) is appended to each message. This helps in detecting any tampering or alteration of the data during transmission.

Unit-5
Service Layer Protocols and Security

Security in IEEE 802.15.4

3. **Security Control Fields**: The MAC frame includes security control fields that specify the type of protection used, such as encryption and replay protection. These fields help in managing security policies and ensuring secure communication.
4. **Key Management**: While IEEE 802.15.4 specifies the encryption algorithm, it does not define key management or authentication policies. These aspects are typically handled by higher-layer protocols like Zigbee.
5. **Security Overhead**: Implementing security features does introduce some overhead, which can affect the performance of the network, especially in resource-constrained environments.

Unit-5
Service Layer Protocols and Security

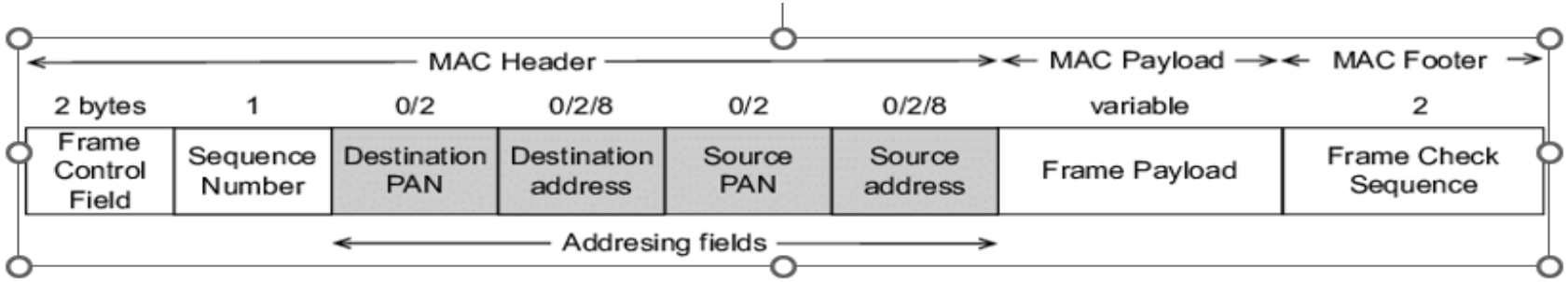
Security in IEEE 802.15.4

Security in the IEEE 802.15.4 MAC layer is designed to provide **confidentiality, integrity, authenticity**, and replay protection for transmitted data. It supports lightweight and efficient security mechanisms to suit resource-constrained devices, making it ideal for low-power wireless networks like IoT and sensor networks.

Unit-5

Service Layer Protocols and Security

Security in IEEE 802.15.4



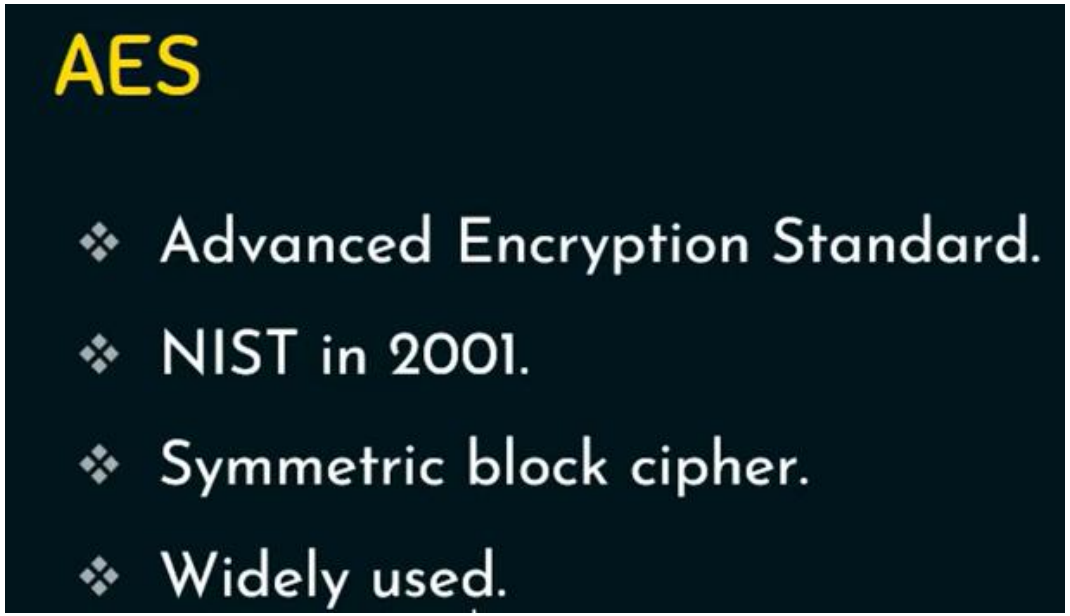
General IEEE 802.15.4 MAC Frame Format

Security:

- 128 bit AES
- Message Integrity code
- Frame Control bit set to 1
- Auxiliary Security Header

Unit-5
Service Layer Protocols and Security

Security in IEEE 802.15.4



- ✓ National Institute of Standards and Technology (NIST) is a U.S. Department of Commerce agency that works on networking and cybersecurity in several ways

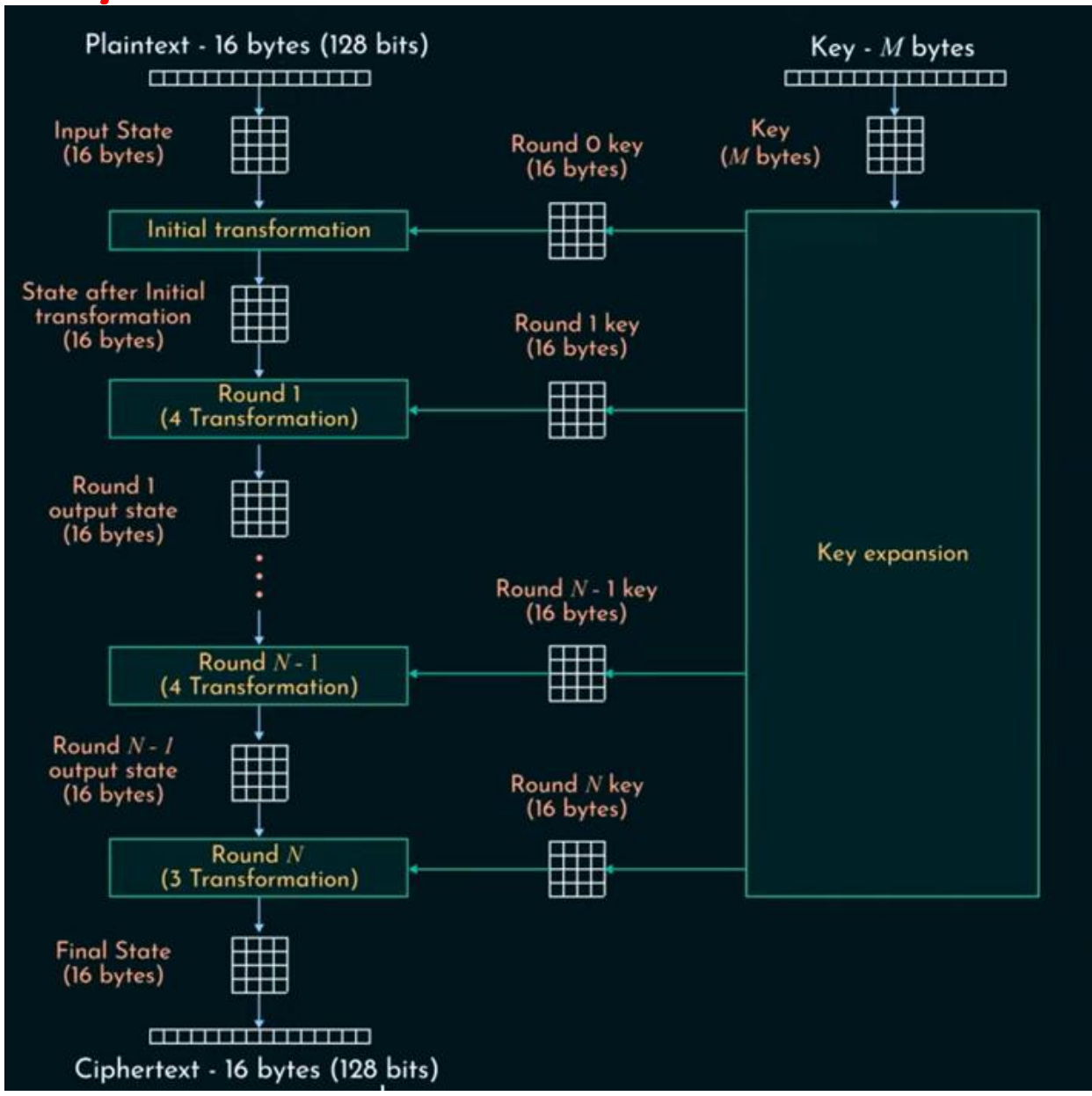
Unit-5

Service Layer Protocols and Security

Security in IEEE 802.15.4

AES Structure

No. of rounds	Key size (in bits)
10	128
12	192
14	256



Unit-5
Service Layer Protocols and Security

Security in IEEE 802.15.4

AES Parameters

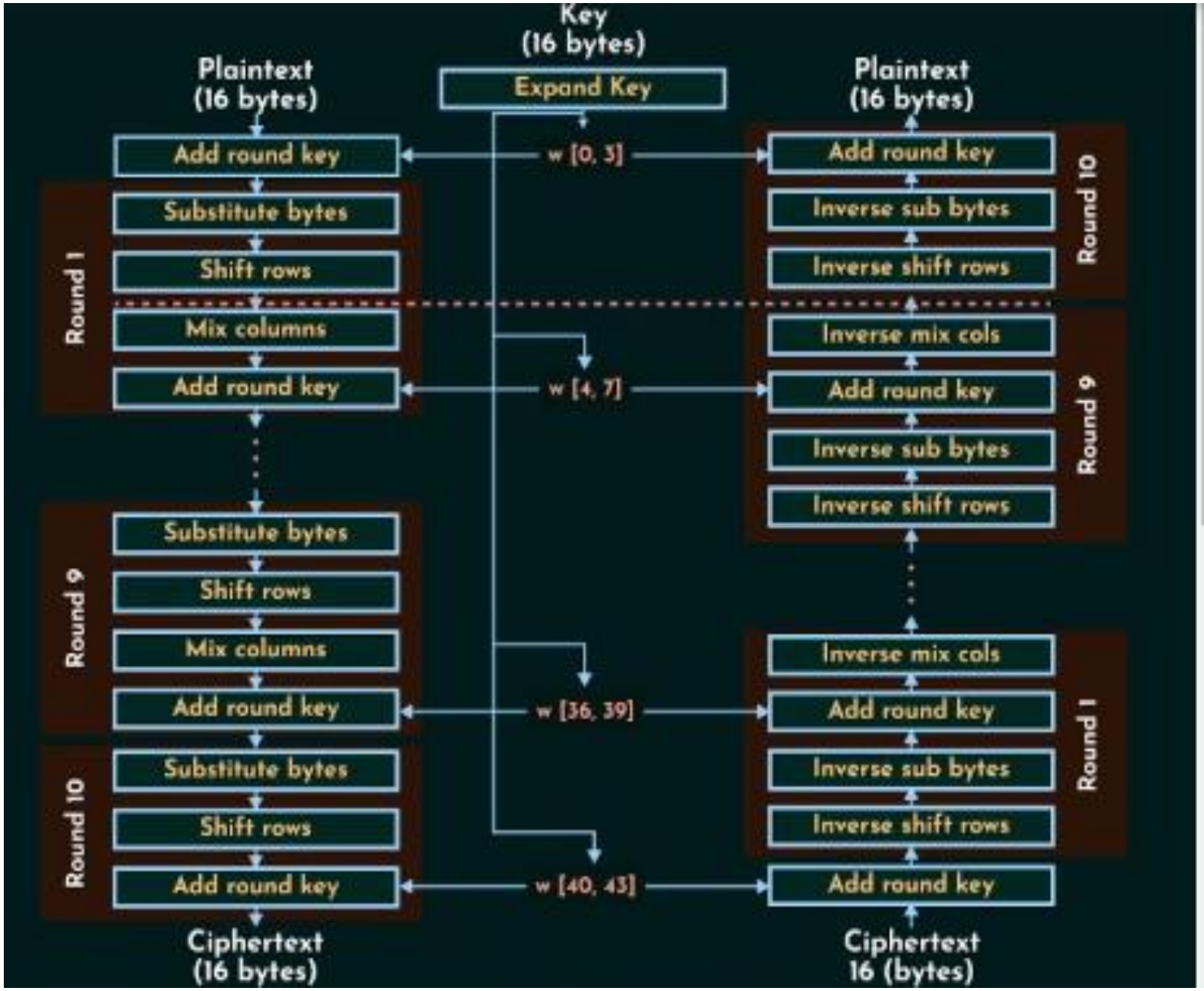
	AES-128	AES-192	AES-256
Key Size	128	192	256
Plaintext Size	128	128	128
Number of rounds	10	12	14
Round Key Size	128	128	128

Unit-5

Service Layer Protocols and Security

Security in IEEE 802.15.4

AES Encryption & Decryption



Unit-5

Service Layer Protocols and Security

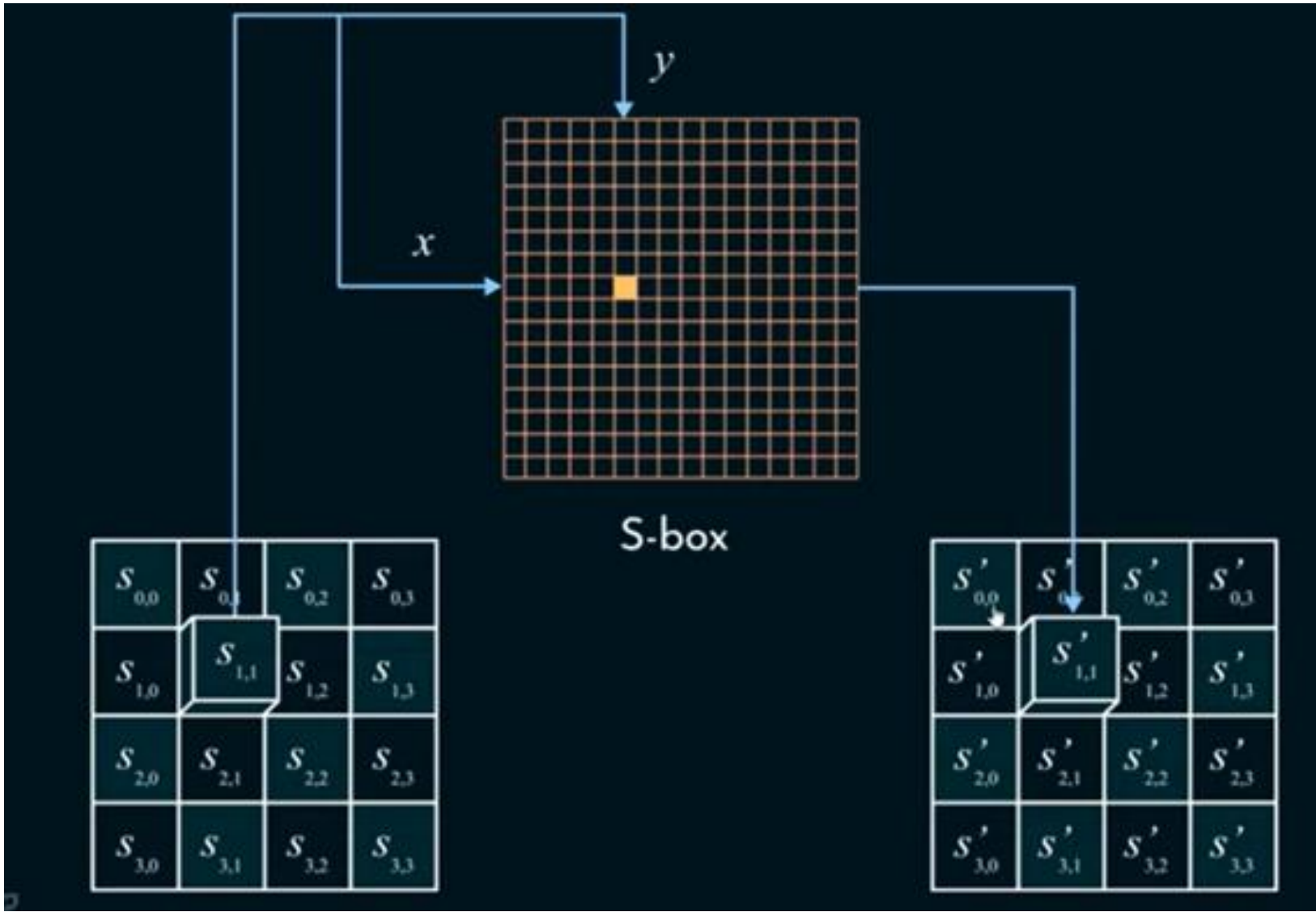
Security in IEEE 802.15.4



Unit-5
Service Layer Protocols and Security

Security in IEEE 802.15.4

Substitute Bytes



Unit-5
Service Layer Protocols and Security

Security in IEEE 802.15.4

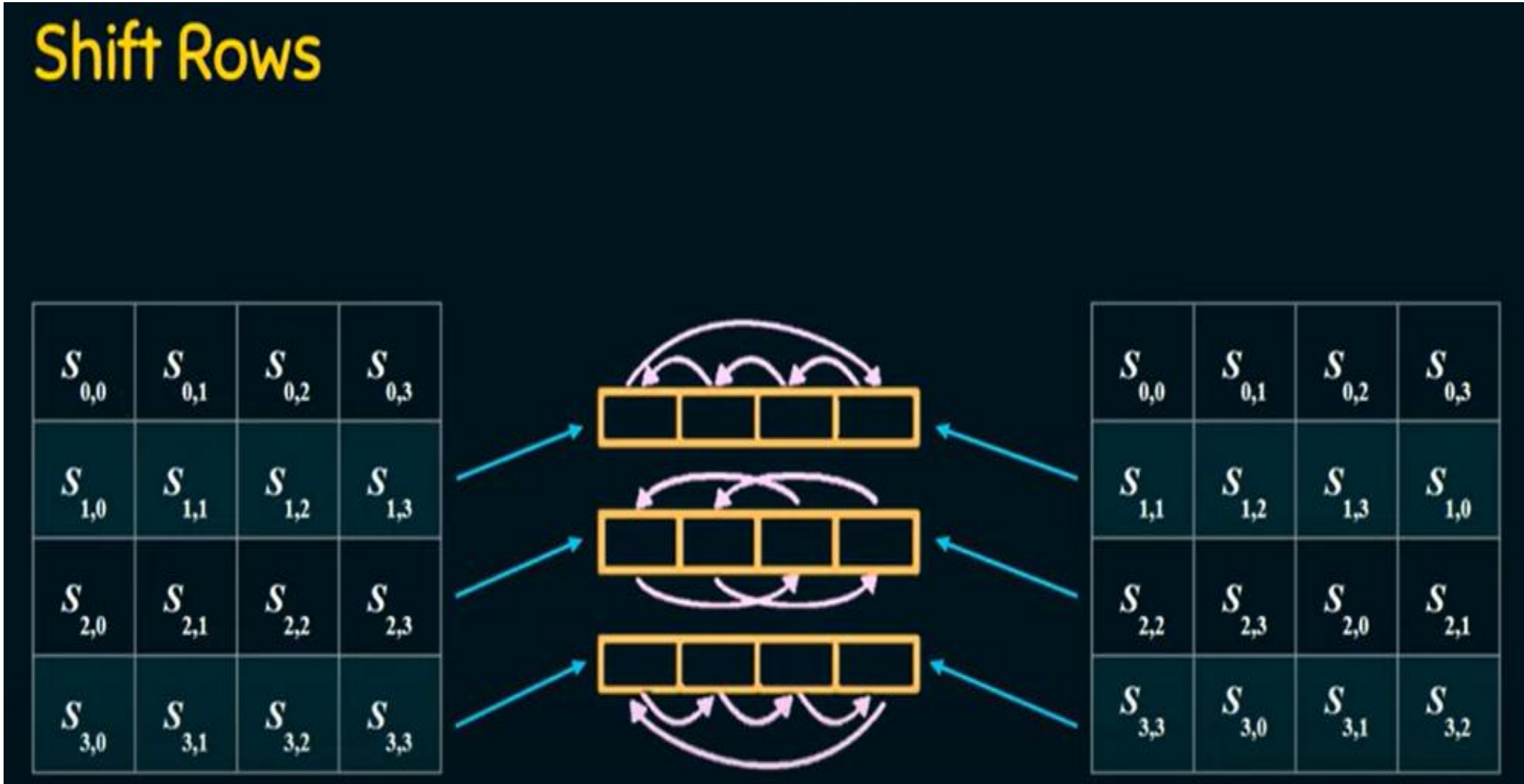
Substitute Bytes

S-Box →

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

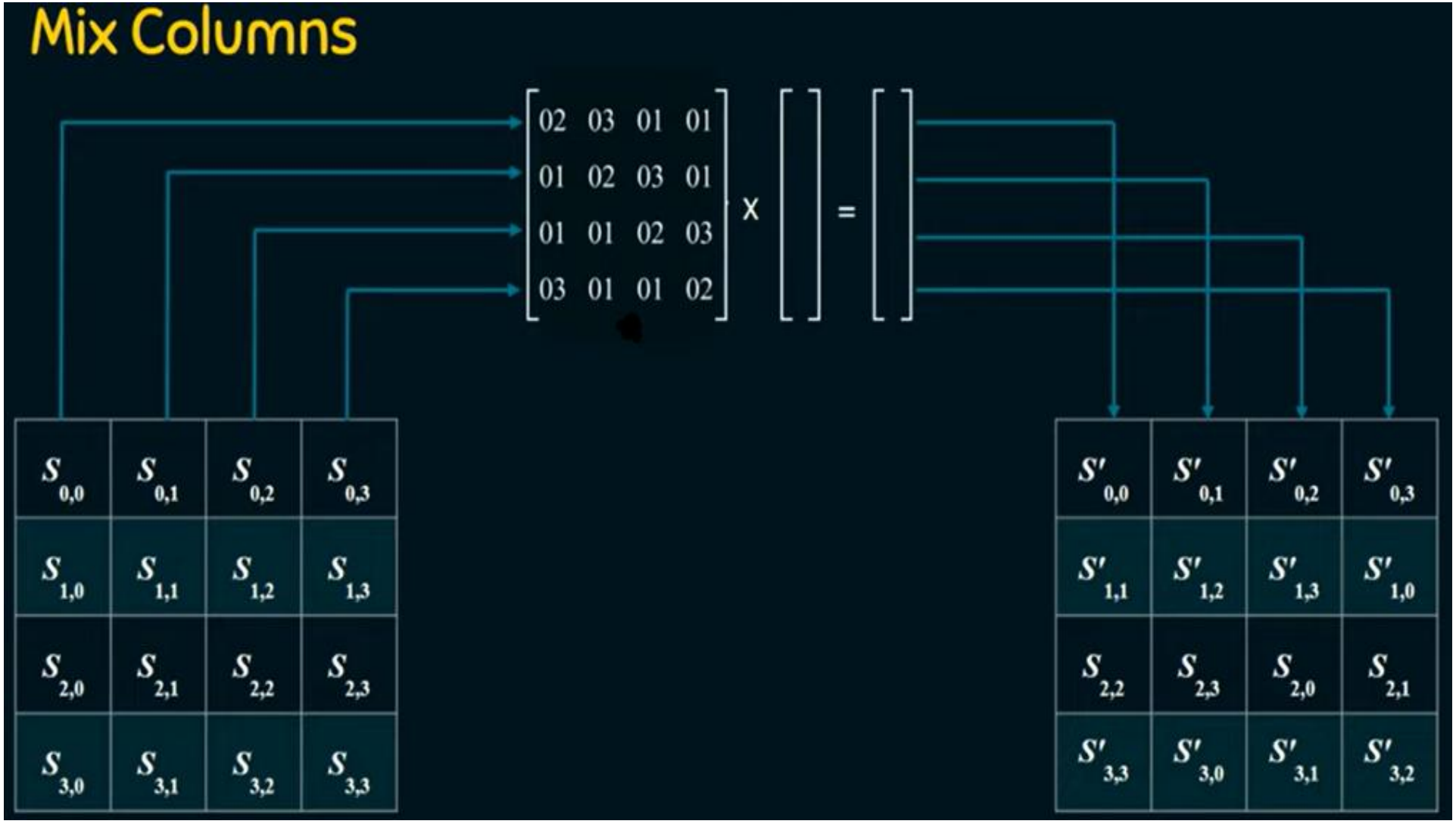
Unit-5
Service Layer Protocols and Security

Security in IEEE 802.15.4



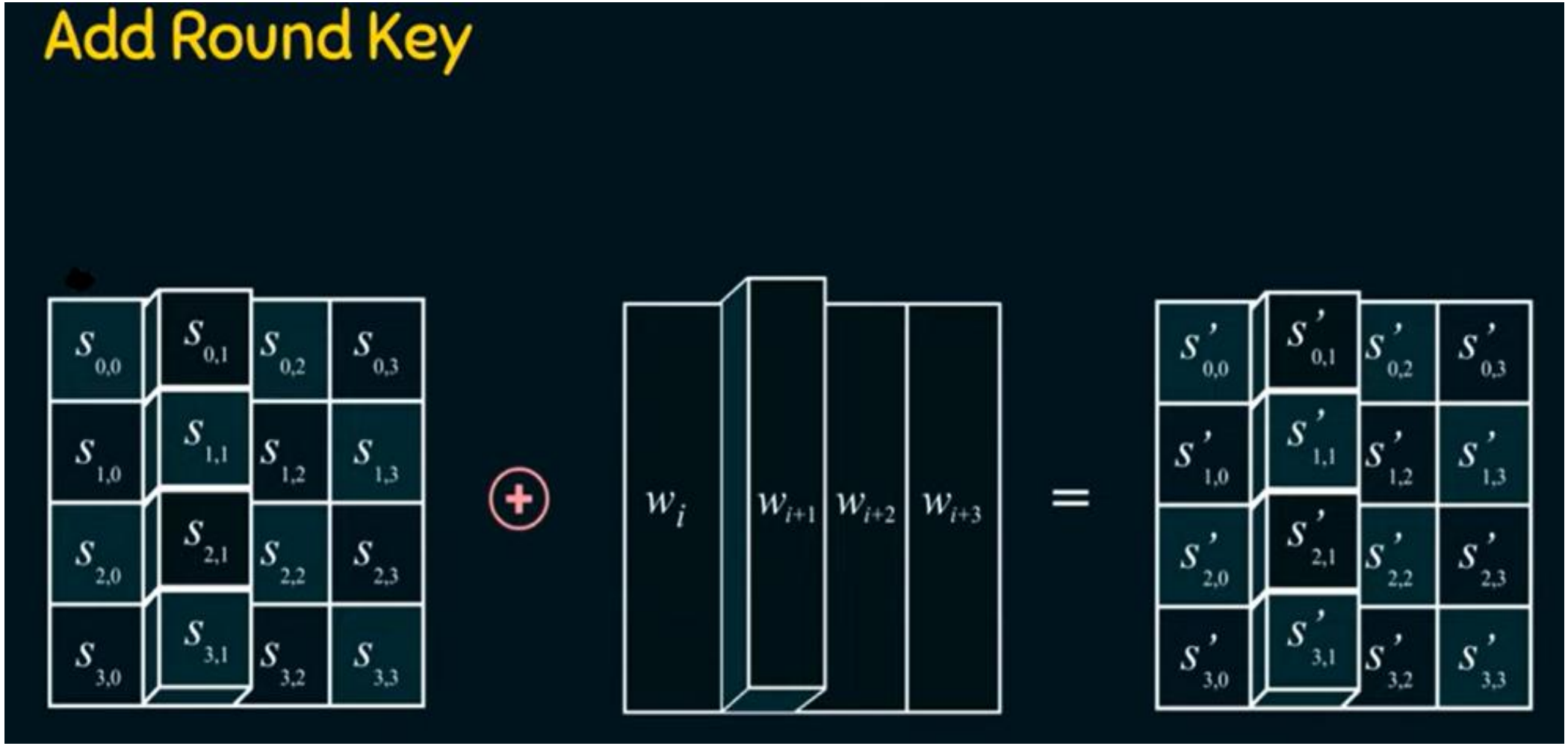
Unit-5
Service Layer Protocols and Security

Security in IEEE 802.15.4



Unit-5
Service Layer Protocols and Security

Security in IEEE 802.15.4



Unit-5

Service Layer Protocols and Security

Security in IEEE 802.15.4-AES Example

Plaintext – Two One Nine Two

T	w	o		O	n	e		N	i	n	e		T	w	o
54	77	6F	20	4F	6E	65	20	43	69	6E	25	20	54	77	6F

Plaintext in Hex Format

54 77 6F 20 4F 6E 65 20 43 69 6E 25 20 54 77 6F

Encryption Key – Thats my Kung Fu

T	h	a	t	s		m	y		K	u	n	g		F	u
54	68	61	74	73	20	6D	79	20	4B	75	6E	67	20	46	75

Encryption Key in Hex Format

54 68 61 74 73 20 6D 79 20 4B 75 6E 67 20 46 75

Unit-5
Service Layer Protocols and Security

Security in IEEE 802.15.4-AES Example

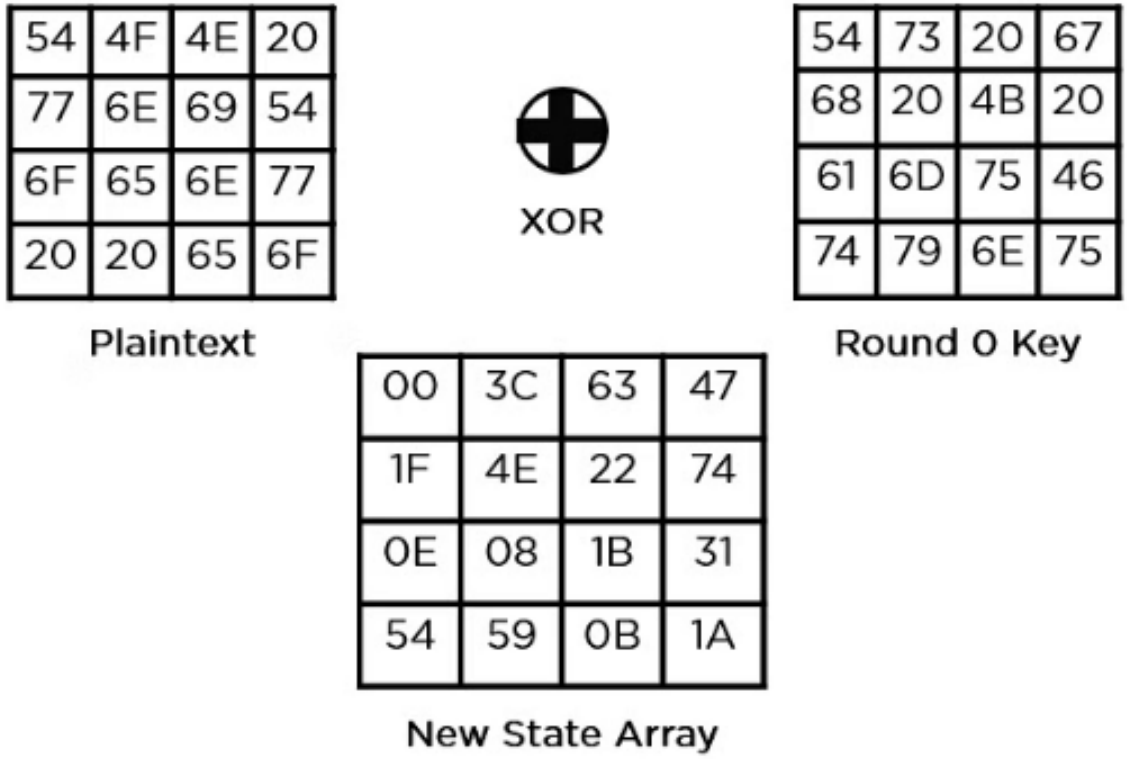
Keys generated for every round

- Round 0: 54 68 61 74 73 20 6D 79 20 4B 75 6E 67 20 46 75
- Round 1: E2 32 FC F1 91 12 91 88 B1 59 E4 E6 D6 79 A2 93
- Round 2: 56 08 20 07 C7 1A B1 8F 76 43 55 69 A0 3A F7 FA
- Round 3: D2 60 0D E7 15 7A BC 68 63 39 E9 01 C3 03 1E FB
- Round 4: A1 12 02 C9 B4 68 BE A1 D7 51 57 A0 14 52 49 5B
- Round 5: B1 29 3B 33 05 41 85 92 D2 10 D2 32 C6 42 9B 69
- Round 6: BD 3D C2 B7 B8 7C 47 15 6A 6C 95 27 AC 2E 0E 4E
- Round 7: CC 96 ED 16 74 EA AA 03 1E 86 3F 24 B2 A8 31 6A
- Round 8: 8E 51 EF 21 FA BB 45 22 E4 3D 7A 06 56 95 4B 6C
- Round 9: BF E2 BF 90 45 59 FA B2 A1 64 80 B4 F7 F1 CB D8
- Round 10: 28 FD DE F8 6D A4 24 4A CC C0 A4 FE 3B 31 6F 26

Unit-5
Service Layer Protocols and Security

Security in IEEE 802.15.4-AES Example

- Add Round Key:



Unit-5
Service Layer Protocols and Security

Security in IEEE 802.15.4

Substitute Bytes

S-Box →

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

Unit-5
Service Layer Protocols and Security

Security in IEEE 802.15.4-AES Example

- Sub-Bytes: It passes the elements through a 16x16 S-Box to get a completely new state array.

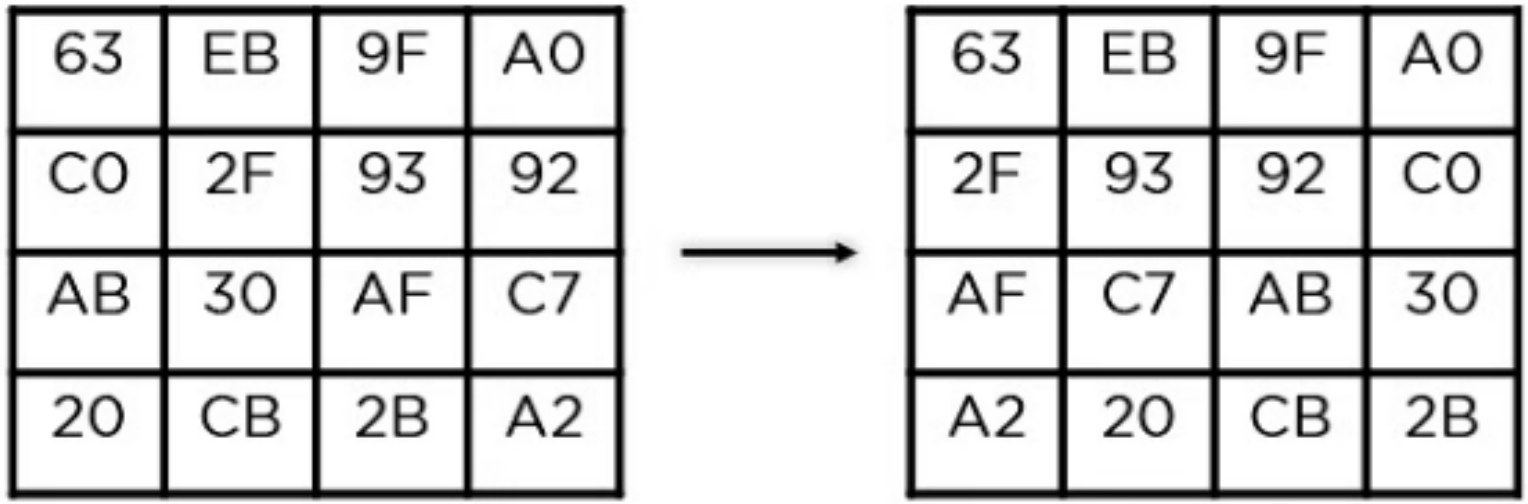
63	EB	9F	A0
C0	2F	93	92
AB	30	AF	C7
20	CB	2B	A2

New State Array

Unit-5
Service Layer Protocols and Security

Security in IEEE 802.15.4-AES Example

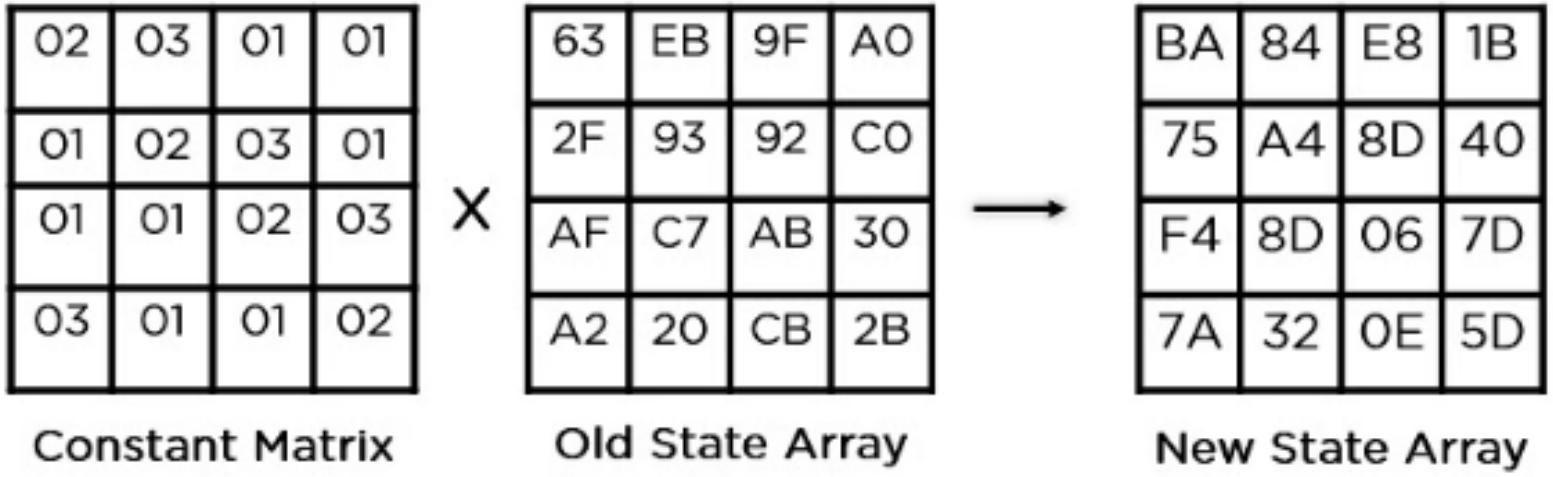
- Shift Rows:



Unit-5
Service Layer Protocols and Security

Security in IEEE 802.15.4-AES Example

- Mix Columns:



Unit-5
Service Layer Protocols and Security

Security in IEEE 802.15.4-AES Example

- Add Round Key:

BA	84	E8	1B
75	A4	8D	40
F4	8D	06	7D
7A	32	0E	5D

Old State Array



XOR

E2	91	B1	D6
32	12	59	79
FC	91	E4	A2
F1	88	E6	93

Round 1 Key

58	15	59	CD
47	B6	D4	39
08	1C	E2	DF
8B	BA	E8	CE

New State Array

Unit-5
Service Layer Protocols and Security

Security in IEEE 802.15.4-AES Example

This state array is now the final ciphertext for this particular round. This becomes the input for the next round. Depending on the key length, you repeat the above steps until you complete round 10, after which you receive the final ciphertext.

Final State Array after Round 10

29	57	40	1A
C3	14	22	02
50	20	99	D7
5F	F6	B3	3A

AES Final Output

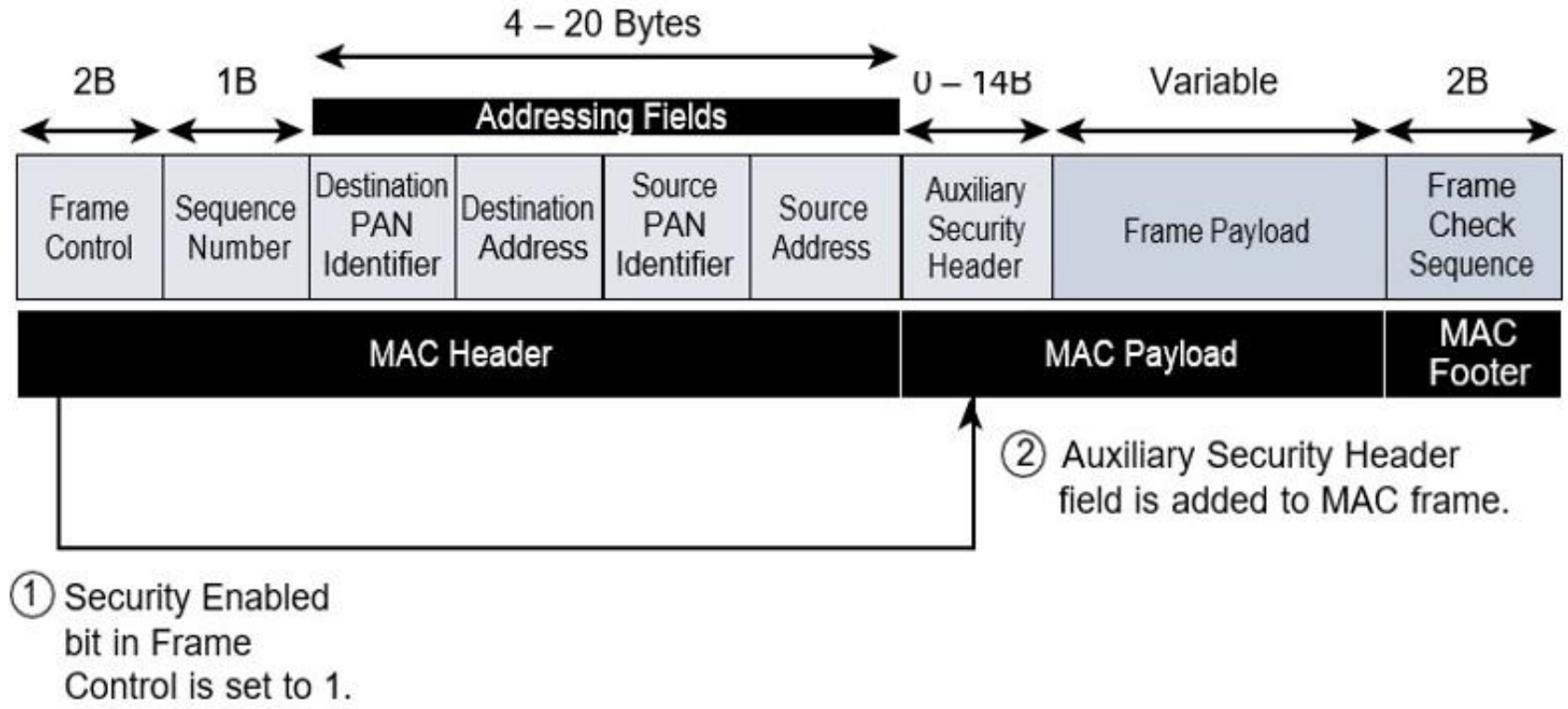
29 C3 50 5F 57 14 20 F6 40 22 99 B3 1A 02 D7 3A ← Cipher Text

Unit-5
Service Layer Protocols and Security

Security in IEEE 802.15.4

Security:

- 128 bit AES
- Message Integrity code
- Frame Control bit set to 1
- Auxiliary Security Header



Unit-5
Service Layer Protocols and Security

Security in IEEE 802.15.4

- The AES algorithm is not only used to encrypt the information but to validate the data which is sent. This concept is called **Data Integrity** and it is achieved using a **Message Integrity Code (MIC)** also named as **Message Authentication Code (MAC)** which is appended to the message. This code ensures integrity of the MAC header and payload data attached.
- It is created encrypting parts of the IEEE MAC frame using the Key of the network, so if we receive a message from a non trusted node we will see that the MAC generated for the sent message does not correspond to the one what would be generated using the message with the current secret Key, so we can discard this message.

Unit-5
Service Layer Protocols and Security

Security in IEEE 802.15.4

- The MAC can have different sizes: 32, 64, 128 bits, however it is always created using the **128b AES** algorithm. Its size is just the bits length which is attached to each frame.
- **Data Security** is performed encrypting the data payload field with the 128b Key.

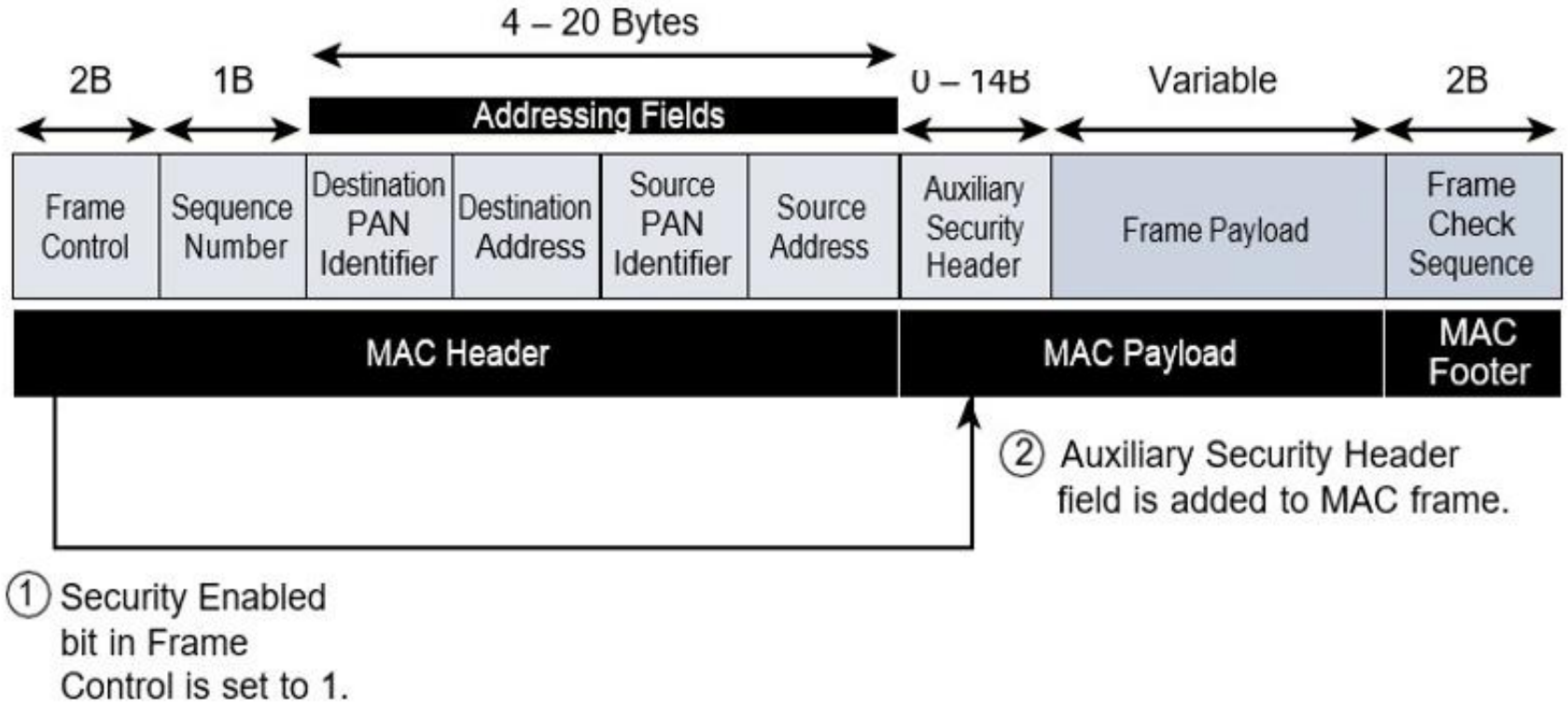
802.15.4 Security Insights:

There are three fields in the IEEE 802.15.4 MAC frame which are related to security issues:

Frame Control (located in the MAC Header)
Auxiliary Security Control (in the MAC Header)
Data Payload (in the MAC Payload field)

Unit-5
Service Layer Protocols and Security

Security in IEEE 802.15.4



Frame Control (located in the MAC Header)
Auxiliary Security Control (in the MAC Header)
Data Payload (in the MAC Payload field)

Unit-5
Service Layer Protocols and Security

Security in IEEE 802.15.4

The **Auxiliary Security Frame** is only enabled if the **Security Enabled** subfield of the **Frame Control Frame** is turned on. This special header has 3 fields:

Security Control (1B) specifies which kind of protection is used.

Frame Counter (4B) is a counter given by the source of the current frame in order to protect the message from replaying protection. For this reason each message has a unique sequence ID represented by this field.

Key Identifier (0-9B) specifies the information needed to know what key we are using with the node we are communicating with.

Unit-5
Service Layer Protocols and Security

Security in 6LoWPAN

6LoWPAN uses IPv6 in Network Layer



Unit-5
Service Layer Protocols and Security

Security in 6LoWPAN







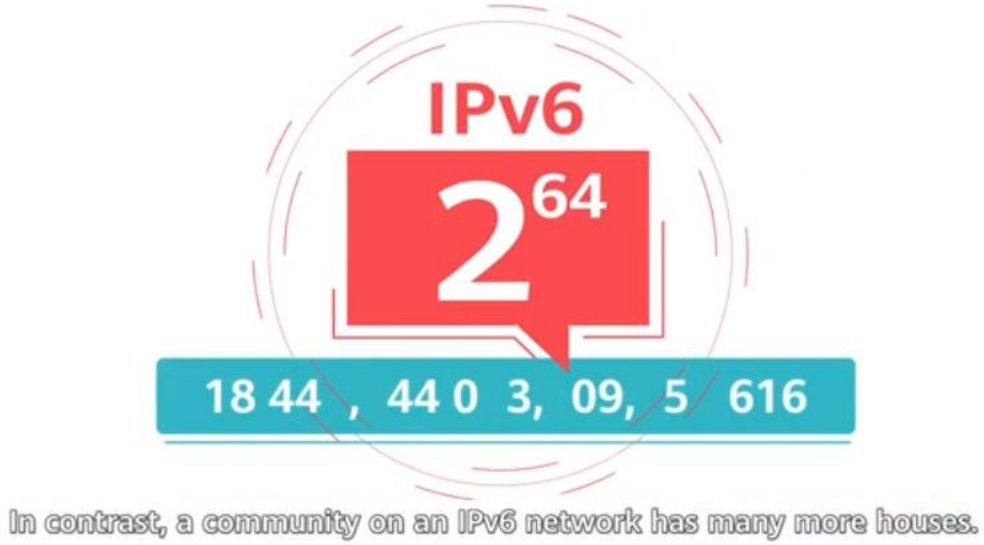
Unit-5
Service Layer Protocols and Security

Security in 6LoWPAN

- First ipv6 makes it more difficult to execute scanning attacks
- If you think of hackers as thieves a scanning attack is their reconnaissance before committing a crime
- A community on an ipv4 network has a limited number of houses or ip addresses, so a thief can go door to door and quickly break into any that are unlocked
- In contrast a community on the ipv6 network has many more houses, even if a super thief were to try one million households per second, it will still take around 500 000 years

Unit-5
Service Layer Protocols and Security

Security in 6LoWPAN



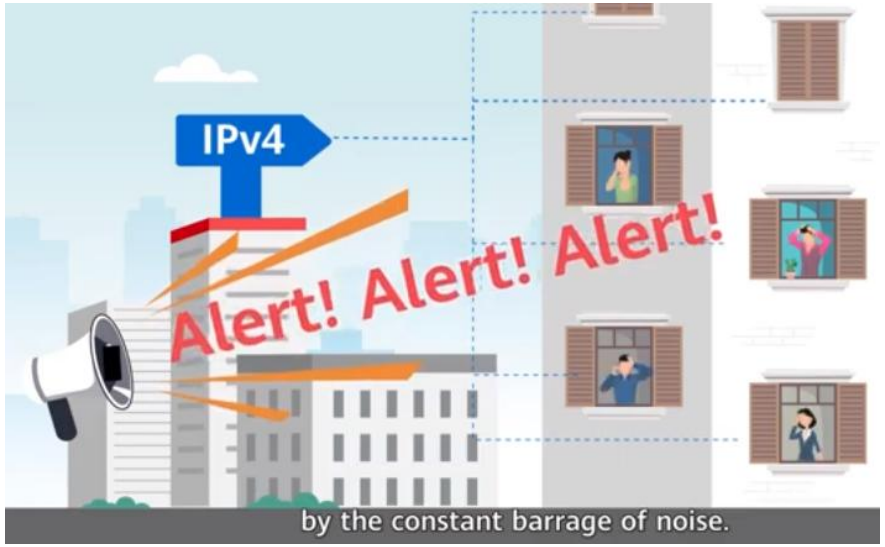




Unit-5
Service Layer Protocols and Security

Security in 6LoWPAN







Unit-5
Service Layer Protocols and Security

Security in 6LoWPAN

- Ipv6 also eliminates broadcast storms,
- Ipv4 broadcasts to an entire community, so what interested households become frustrated by the constant barrage of noise
- Ipv6 does not broadcast to the entire community, only households with common needs communicate with each other.
- This prevents network amplification attacks caused by broadcaster storms

Unit-5
Service Layer Protocols and Security

Security in 6LoWPAN







Unit-5
Service Layer Protocols and Security

Security in 6LoWPAN

- Fragmentation attacks
- Fragmentation refers to a network device that splits a large data packet into smaller pieces and transmits them over a network.
- On an ipv4 network this is akin to a household buying a large piece of furniture online.
- With many couriers involved in the delivery each works out what it can manage and the furniture is then split into separate parts.
- If just one part gets lost on the way the furniture can't be reassembled.
- By contrast ipv6 doesn't allow the furniture to be split up ensuring that all parts are delivered together and in usable condition

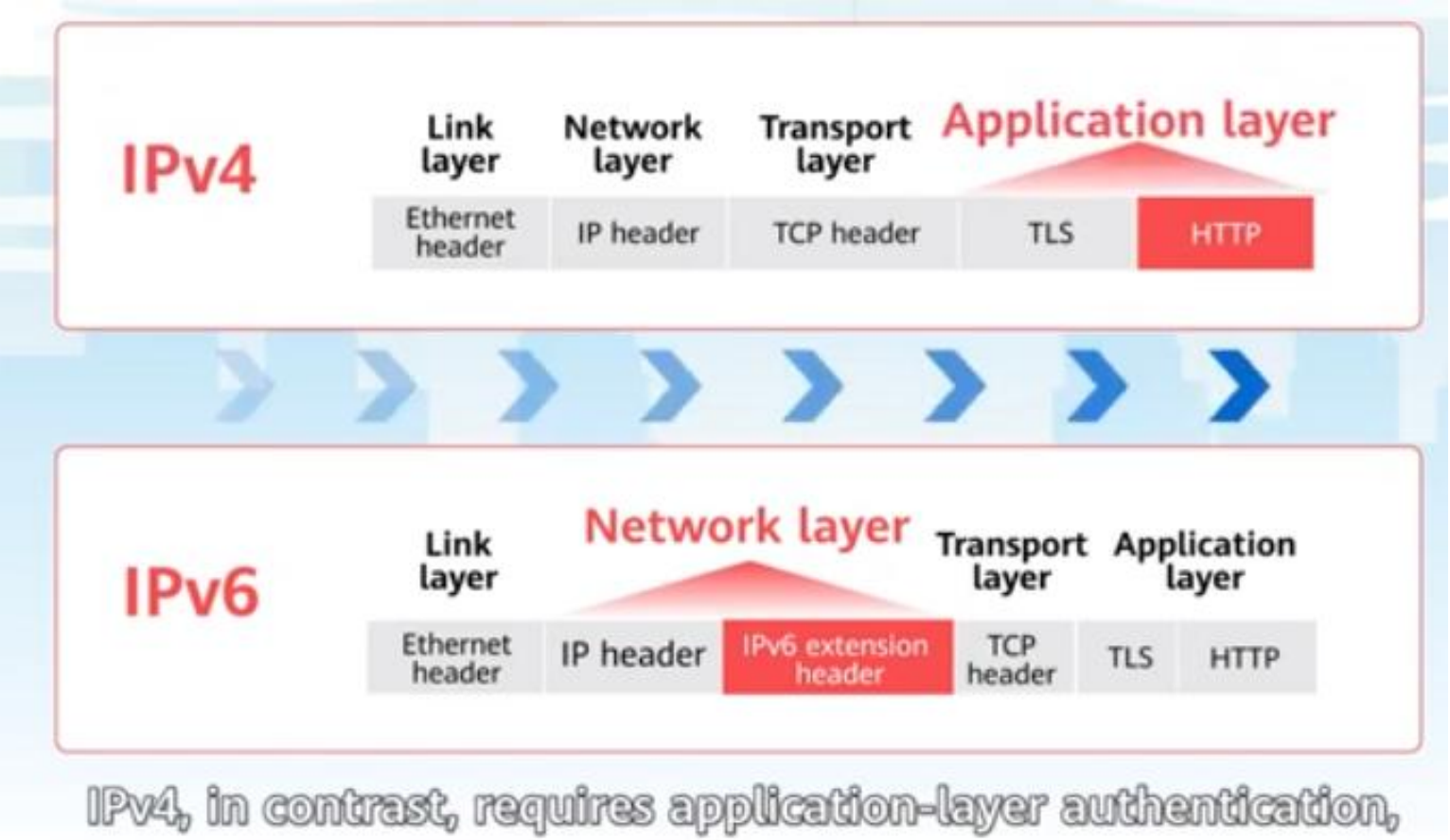
Unit-5
Service Layer Protocols and Security

Security in 6LoWPAN



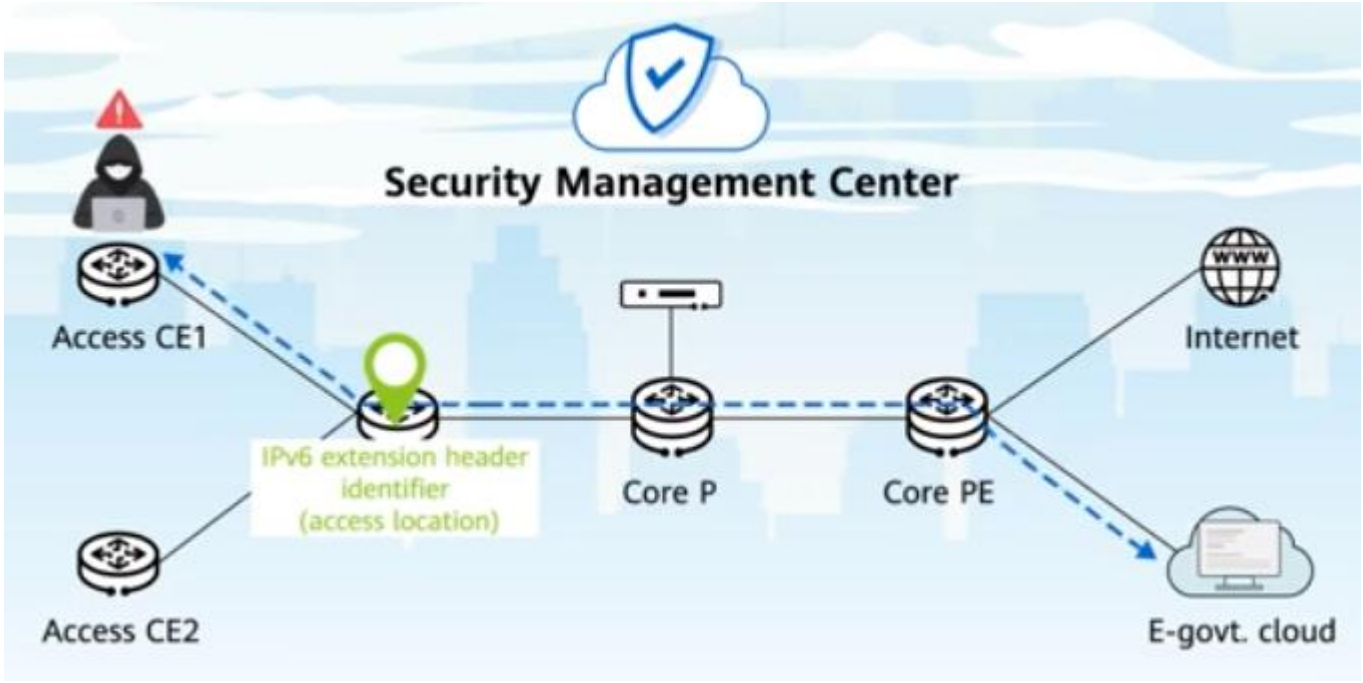
Unit-5
Service Layer Protocols and Security

Security in 6LoWPAN



Unit-5
Service Layer Protocols and Security

Security in 6LoWPAN



• Ipv6 also improves threat blocking efficiency as this extension header can mark the location attribute of traffic.
•When a system detects a threat it quickly locates the source and blocks it

Unit-5
Service Layer Protocols and Security

Security in RPL

RPL (Routing Protocol for Low-Power and Lossy Networks)

RPL is a routing protocol specifically designed for **low-power and lossy networks (LLNs)**, which are typically characterized by constrained devices (e.g., sensors) and unreliable links.

Unit-5
Service Layer Protocols and Security

Security in RPL

- RPL has three basic security modes:

1. Unsecured: In this mode, RPL control messages are sent without any additional security mechanisms.
2. Pre-installed: in this mode, nodes joining a RPL instance have pre-installed keys that enable them to process and generate secured RPL messages.
3. Authenticated: In authenticated mode, nodes have preinstalled keys as in pre-installed mode, but these preinstalled keys may only be used to join a RPL instance as a leaf.

- RPL specifies a security mechanism to prevent its routing control messages and topology from external attackers but it explicitly noted that this security mechanism is optional to implement.

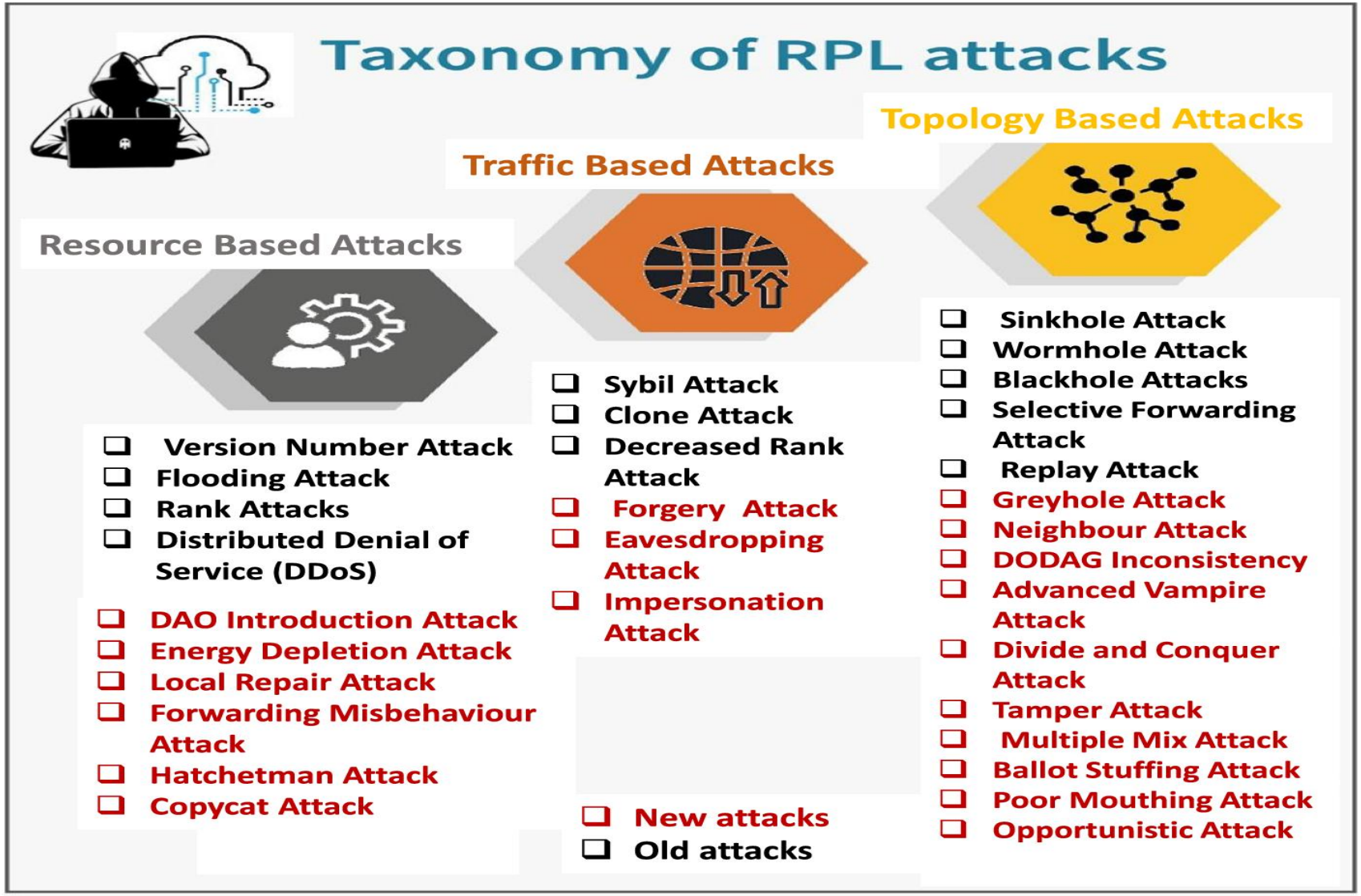
Unit-5
Service Layer Protocols and Security

Security in RPL

- RPL is weak for internal attackers: version number, rank attack, DIS transmission attack, high powered DIO transmission attack, Sybil/Clone attack, Sinkhole/Wormhole attack, redundant local repair attack and selective forwarding attack.
- These attacks spoil the routing topology and/or incur large control overhead which prevents reliable packet delivery and shortens battery lifetime.
- Attacks against RPL are covered in three groups: attacks against resources, attacks on topology and attacks on traffic.
- Attacks on resources aims to deplete the resources of nodes in the network by causing unnecessary operations.
- Attacks on topology affect the construction of RPL topology in a non-optimal way or lead to the isolation of some nodes from the topology.
- Attacks on Traffic : analyze network traffic in order to implement more complex attacks.

Unit-5
Service Layer Protocols and Security

Security in RPL



Unit-5
Service Layer Protocols and Security

Security in RPL

RPL's Security Attacks: Packet Dropping Attacks

- Packet Dropping attacks include: Blackhole, Sinkhole and Selective Fowarding Attack (Greyhole).
- In Blackhole attack a malicious parent node drops all it's children nodes packets.
- Sinkhole is a variation of the Blackhole attack where a malicious node facilitates the attack by performing a rank decrease attack.
- Selective Forwarding attack is another variation of the Blackhole attack where a malicious node drops packets selectively.
- Selective Forwarding attack is further divided:
- SFA-I: Also called Neglect and Greed SFA attack.
- The attacking node fully participates in the transmission and exchange of RPL control packets such as DIS, DIO, DAO and DAO-ACK, and may even acknowledge the reception of the packets to the sender but refuses to forward data packets

Unit-5
Service Layer Protocols and Security

Security in RPL

RPL's Security Attacks: Packet Dropping Attacks

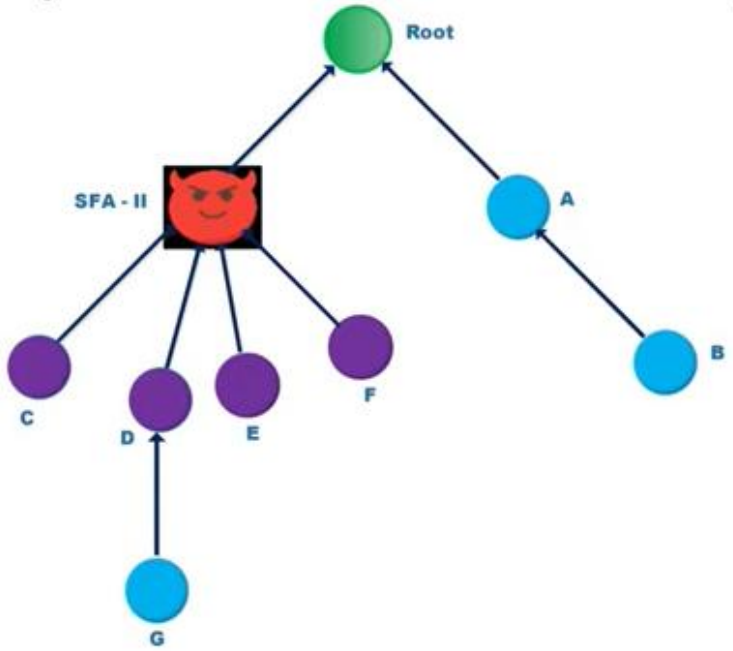


Figure 6: SFA-I

- SFA-II: An attacking node(s) drop a packet that originates from a particular node or group of nodes.

Unit-5
Service Layer Protocols and Security

Security in RPL

RPL's Security Attacks: Packet Dropping Attacks

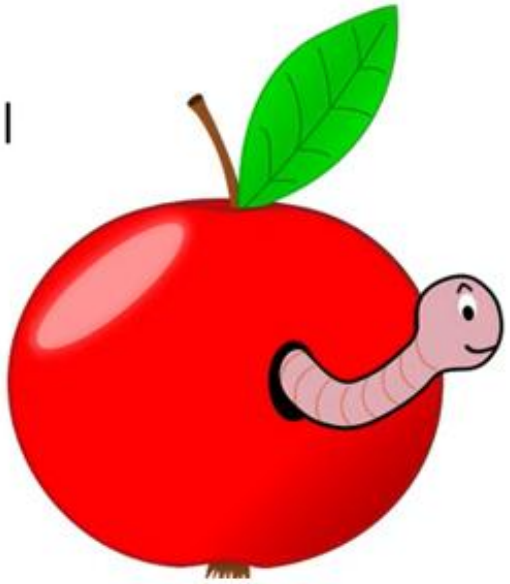
- Multi-Point-to-Point (MP2P) (or many-to-one); (ii) Point-To-Multi-Point (P2MP) (or one-to-many); (iii) Point-To-Point (P2P) (or one-to-one) are the three communication paradigms enabled in RPL.
- SFA-III can exploit these features of the RPL to execute its attacks.
- SFA-IV: The SFA here creates confused routing information between sensor nodes by delaying the packets passing through it.

Unit-5
Service Layer Protocols and Security

Security in RPL

RPL's Security Attacks: Wormhole Attack

- A wormhole is an out of band connection between two nodes using wired or wireless links.
- This attack takes place by creating a tunnel between the two attackers and transmitting the traffic through it.
- Wormholes can be used to forward packets faster than via normal paths.

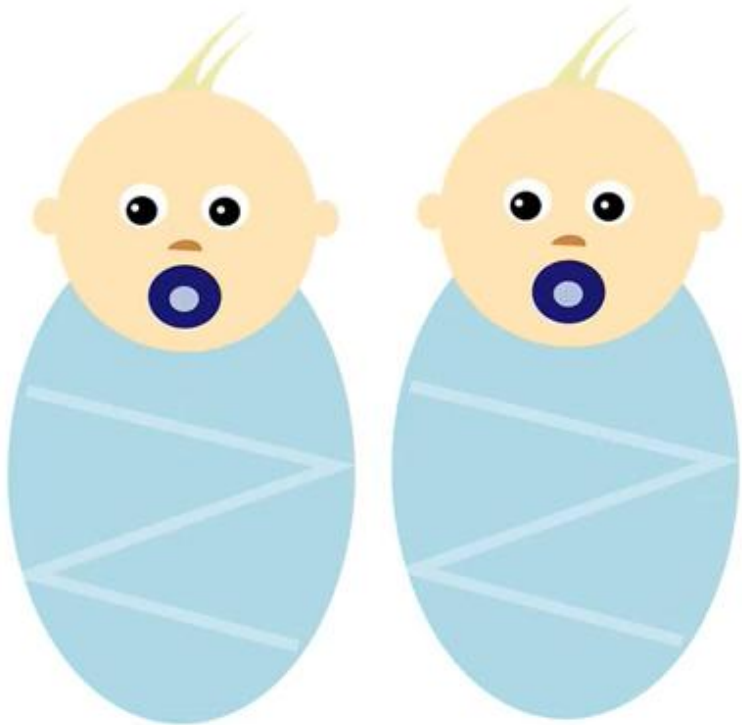


Unit-5
Service Layer Protocols and Security

Security in RPL

RPL's Security Attacks : Clone ID

- A malicious spoofs another node in the network.
- This is done by exploiting the DIS packet.



Unit-5
Service Layer Protocols and Security

Security in RPL

RPL's Security Attacks : Sybil Attack

- In a Sybil attack an attacker uses several logical entities on the same physical node.
- Sybil attacks can be used to take control over large parts of a network without deploying physical nodes.
- DIS transmission mechanism can be exploited by an adversary to attack the network as well.
- If the malicious node generates and multicasts a large number of DIS messages piggy backed with different fictitious identities.

Unit-5
Service Layer Protocols and Security

Security in RPL

RPL's Security Attacks : DIS Flooding Attack

- In RPL, DIS messages are used by nodes to join the network.
- Node sends DIS messages to its neighbor nodes to get the routing information.
- RFC 6650 does not specify the time interval for DIS transmission.
- The DIS transmission interval may vary with different RPL implementations.
- A new node continuously transmits DIS messages with a fixed interval of time until it receives a DIO message from any of its neighbor nodes.
- In RPL, a node can send either multicast or unicast DIS messages.
- Based on the nature of transmission (i.e. multicast or unicast), the receiver node responds in the following manner:

Unit-5
Service Layer Protocols and Security

Security in RPL

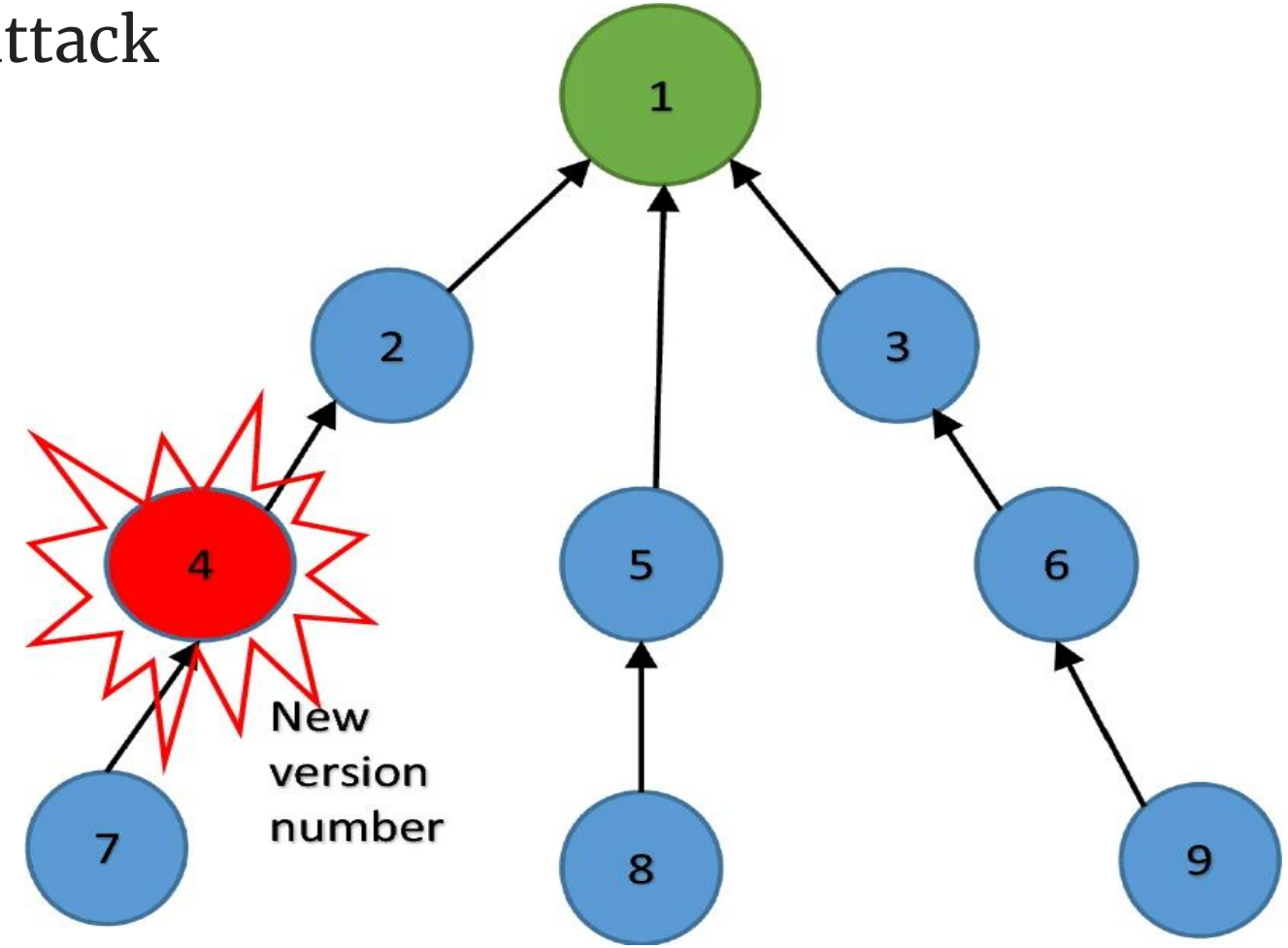
RPL's Security Attacks : DIS Flooding Attack (cont.)

- Case 1: On receiving a multicast DIS, the receiver node resets its Trickle Timer and sends multicast DIO messages containing the latest routing information.
- Case 2: On receiving a unicast DIS, the receiver node directly sends DIO message to the sender node without resetting its Trickle Timer.
- With respect to the type of DIS transmission (multicast or unicast), DIS attacks also fall into two categories: unicast DIS attack and multicast DIS attack.
- In multicast DIS flooding attack, an attacker sends multicast DIS messages to its neighboring nodes frequently.
- On receiving a multicast DIS message, the neighbors within the range of attacker are forced to believe that there is some inconsistency present in the network.

Unit-5
Service Layer Protocols and Security

Security in RPL

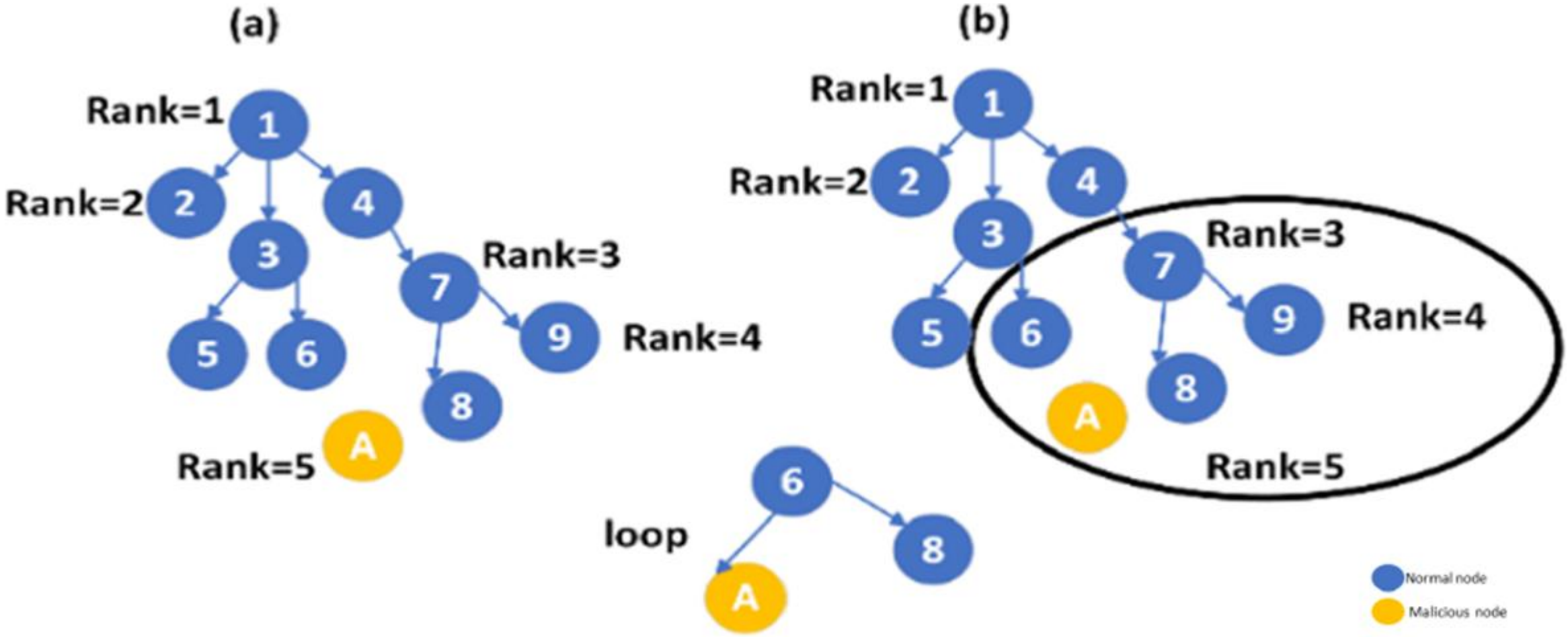
Version number attack



Unit-5
Service Layer Protocols and Security

Security in RPL

Rank attack



Unit-5
Service Layer Protocols and Security

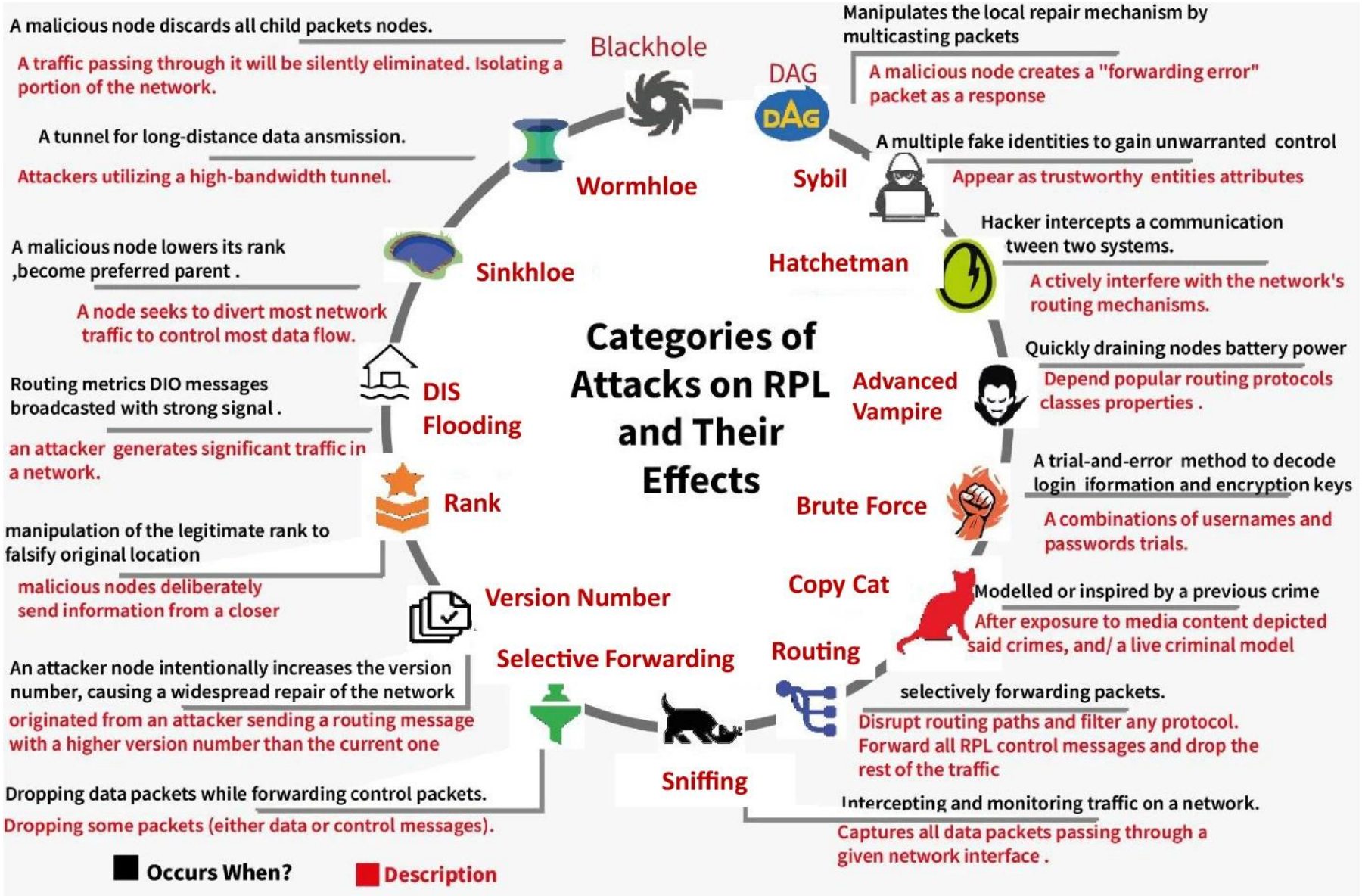
Security in RPL

The security of RPL can be improved by using various security mechanisms, including:

- **Authentication**: Verify the identity of each device before accepting control messages
- **Encryption**: Protect the confidentiality of control messages
- **Secure neighbor discovery protocols**: Use protocols to discover neighbors securely
- **Tamper-resistant rank calculation algorithms**: Use algorithms that are resistant to tampering
- **Continuous monitoring**: Monitor the network for suspicious activities in real-time
- **Anomaly detection**: Use techniques to identify and respond to suspicious activities in real-time

Unit-5
Service Layer Protocols and Security

Security in RPL



Unit-5
Service Layer Protocols and Security

Security in RPL

Attacks	Defense mechanism range	Examples of defense mechanisms
Flooding attacks (FA)	High	- Traffic filtering, rate limiting, anomaly detection, IDS, and firewalls
Replay attacks (RA)	Medium	- Secure protocols and timestamping are necessary. Further actions can be required based on the particular network
Sinkhole attacks (SH)	Medium	- Route validation, reputation systems, blacklisting, traffic redirection,and IDS
Wormhole attacks (WH)	Low	- Protocol patching, secure routing protocols, network monitoring, and IDS prevent exploitation
Denial–of–Service (DoS) Attacks	High	- Traffic filtering, rate limiting, resource monitoring, IDS, firewalls, load balancing
Spoofing attacks (SA)	Medium	- IP address validation, MAC address filtering, data authentication (e.g., digital signatures), secure protocol
Tampering Attacks	Medium	- Data encryption, MA codes (MACs), digital signatures, IDS, data integrity checks
Hatchetman Attack (HA)	Medium	- Secure reputation systems, DAO membership filtering
Sybil attack	High	- Proof–of–stake consensus mechanisms, CAPTCHAs, identity verification
Energy depletion attacks (EDA)	medium	- Low–power communication protocols

Unit-5
Service Layer Protocols and Security

Security in RPL

Decentralized Autonomous Organization (DAO)	Medium	-Secure coding practices, smart contract audits, DAO voting mechanisms
Forwarding Misbehavior attacks (FMA)	Medium	- Secure routing protocols, anomaly detection systems (ADS)
Copycat attacks (CA)	Medium	- Secure communication protocols (e.g., TLS/SSL), digital signatures
Clone attack	medium	-Device authentication (e.g., MAC address filtering), ADS
DECREASE rank attack (DR)	Medium	- Secure ranking algorithms, reputation systems
Greyhole attacks (GH)	Medium	-Traffic analysis, ADS
Neighbour attacks (NA)	Medium	- Neighbor discovery protocols, ADS
Eavesdropping	Medium	- Data encryption (e.g., AES)

Unit-5
Service Layer Protocols and Security

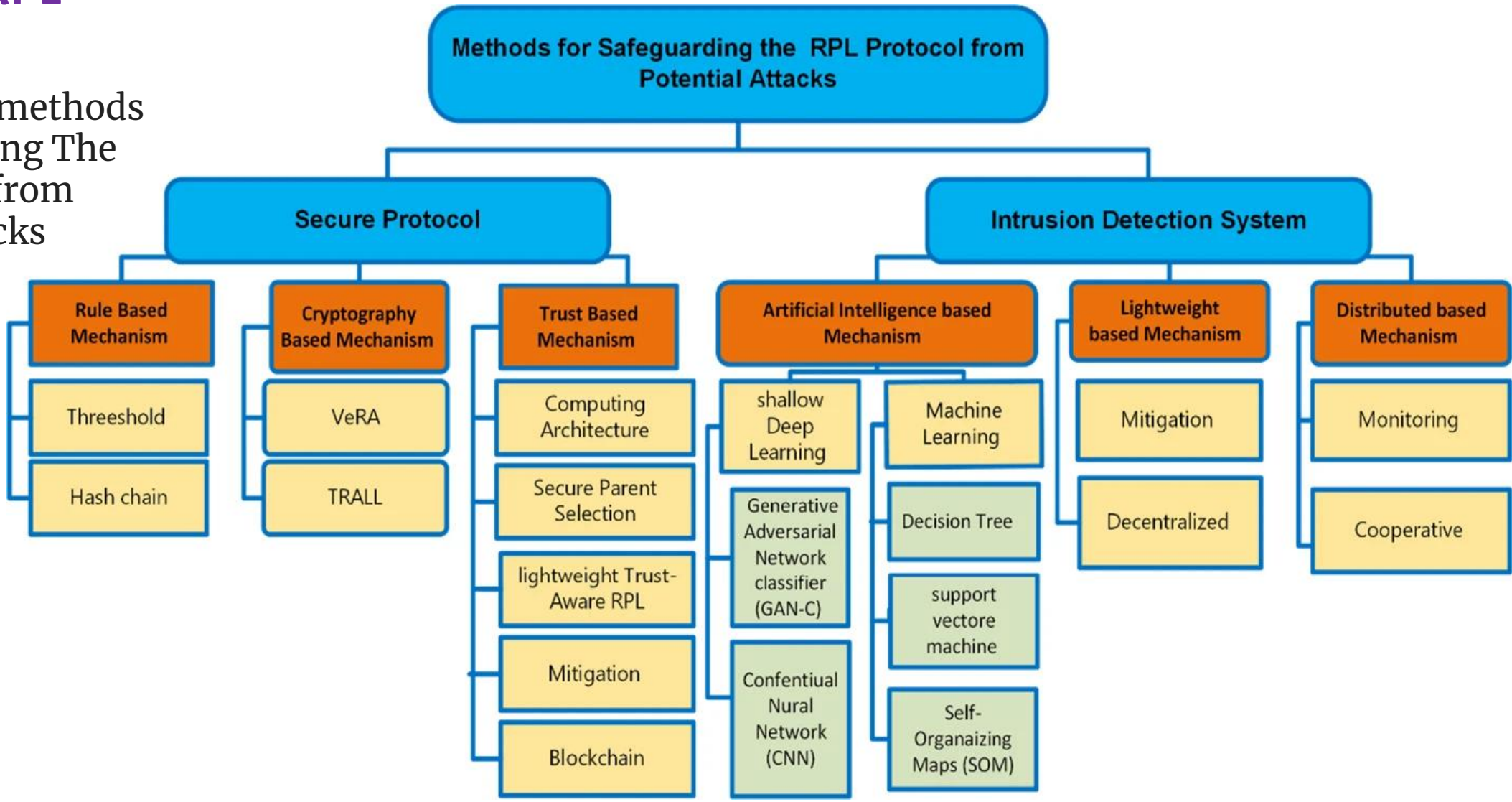
Security in RPL

Rank inconsistency (RI)	Medium	- Secure ranking algorithms, reputation systems
Advanced Vampire Attack (AVA)	Medium	- A complex attack that combines multiple techniques. A combination of defense mechanisms against Sybil attacks, DoS attacks, and resource depletion attacks is needed.
Version Number Attack (VNA)	High	- Secure RPL implementations, ADS

Unit-5
Service Layer Protocols and Security

Security in RPL

Taxonomy of methods for safeguarding The RPL protocol from potential attacks



Unit-5

Service Layer Protocols and Security

Security in Application Layer

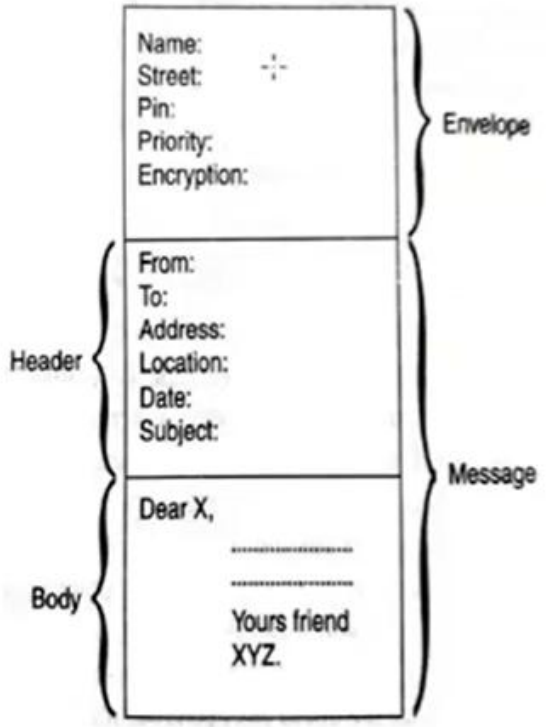
The application layer is the part of this model that humans actually interact with, facilitating communication between a person and a computer over a network. Application layer security simply means protecting this layer and the users who are interacting with it.

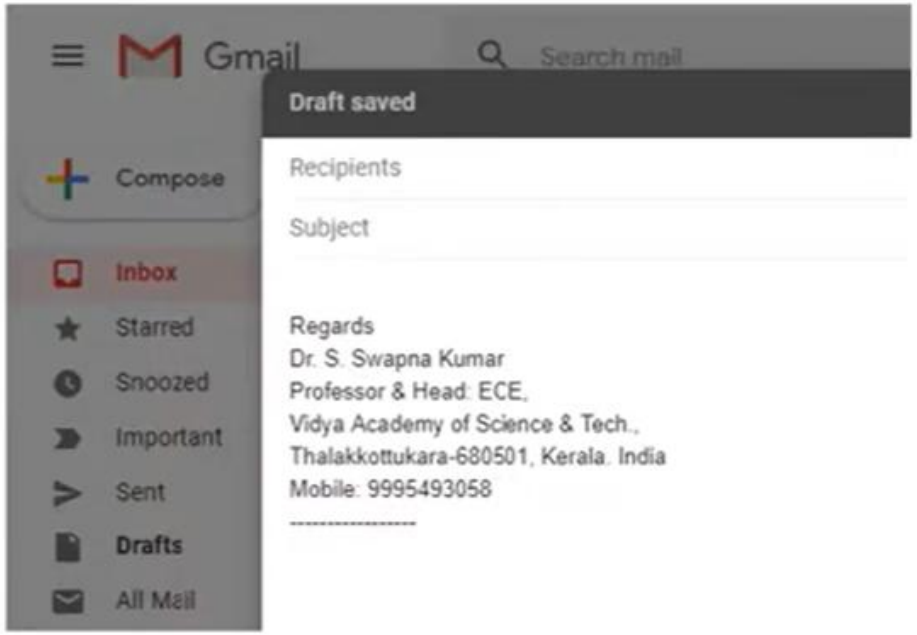
Unit-5

Service Layer Protocols and Security

Security in Application Layer

E-mail format



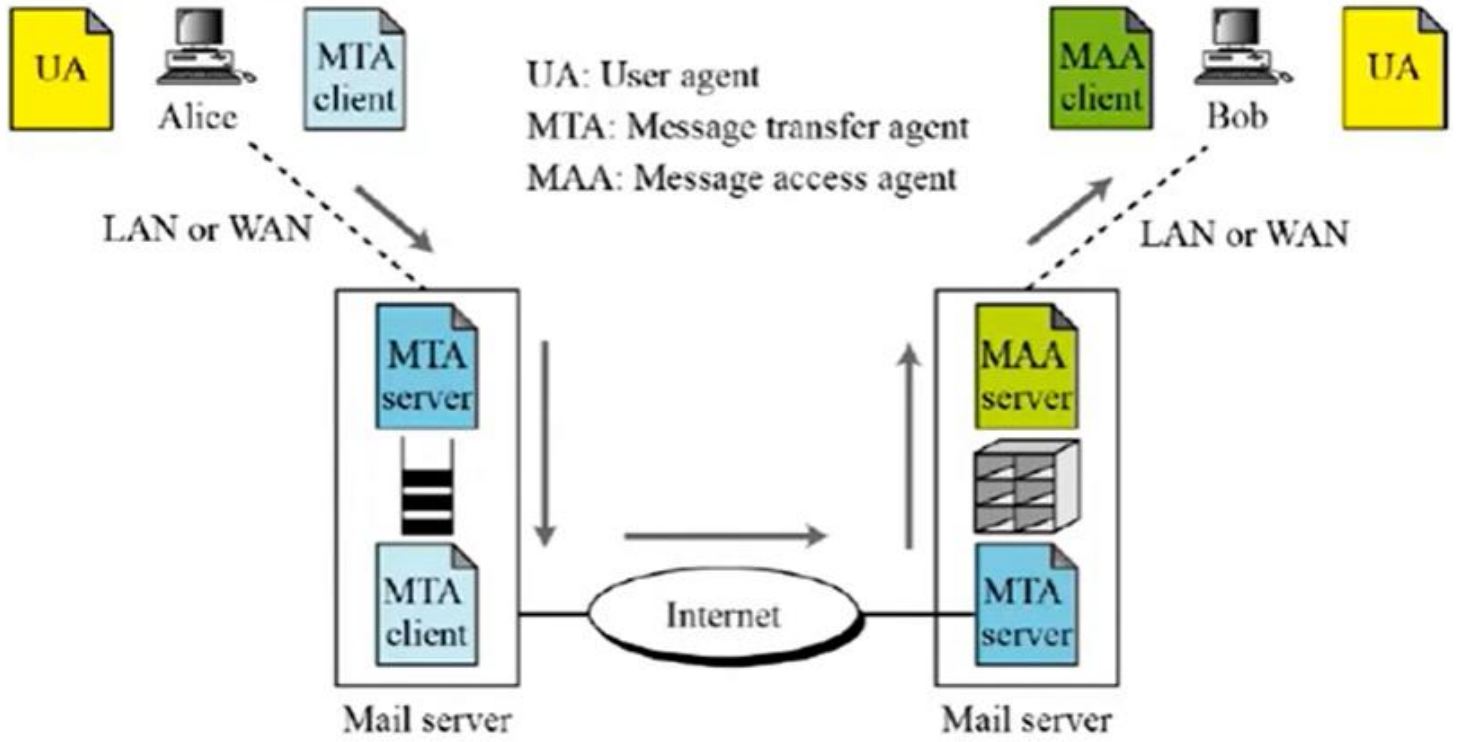


Unit-5

Service Layer Protocols and Security

Security in Application Layer

E-Mail architecture

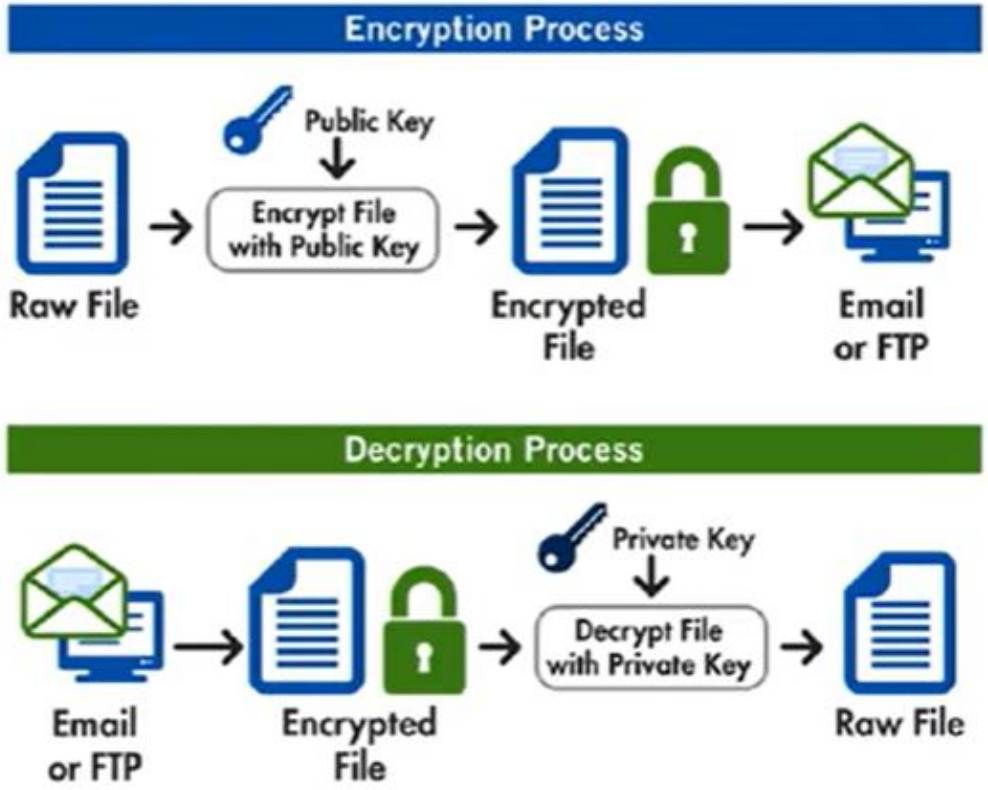


Unit-5

Service Layer Protocols and Security

Security in Application Layer

- Pretty Good Privacy (PGP) can be used to create a secure e-mail message or to store a file securely for future retrieval.



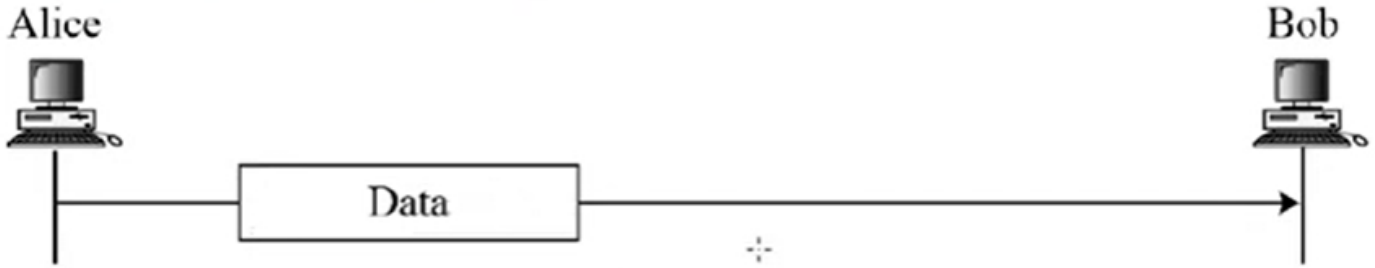
Unit-5

Service Layer Protocols and Security

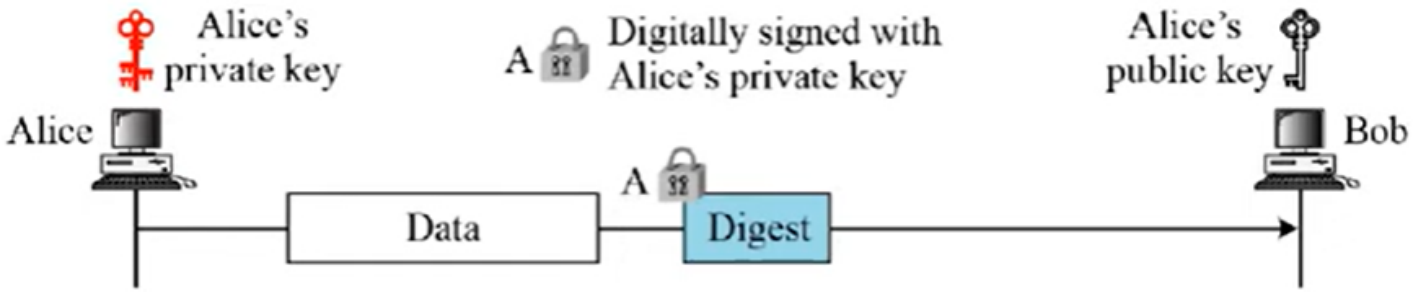
Security in Application Layer

Scenario

A plaintext message



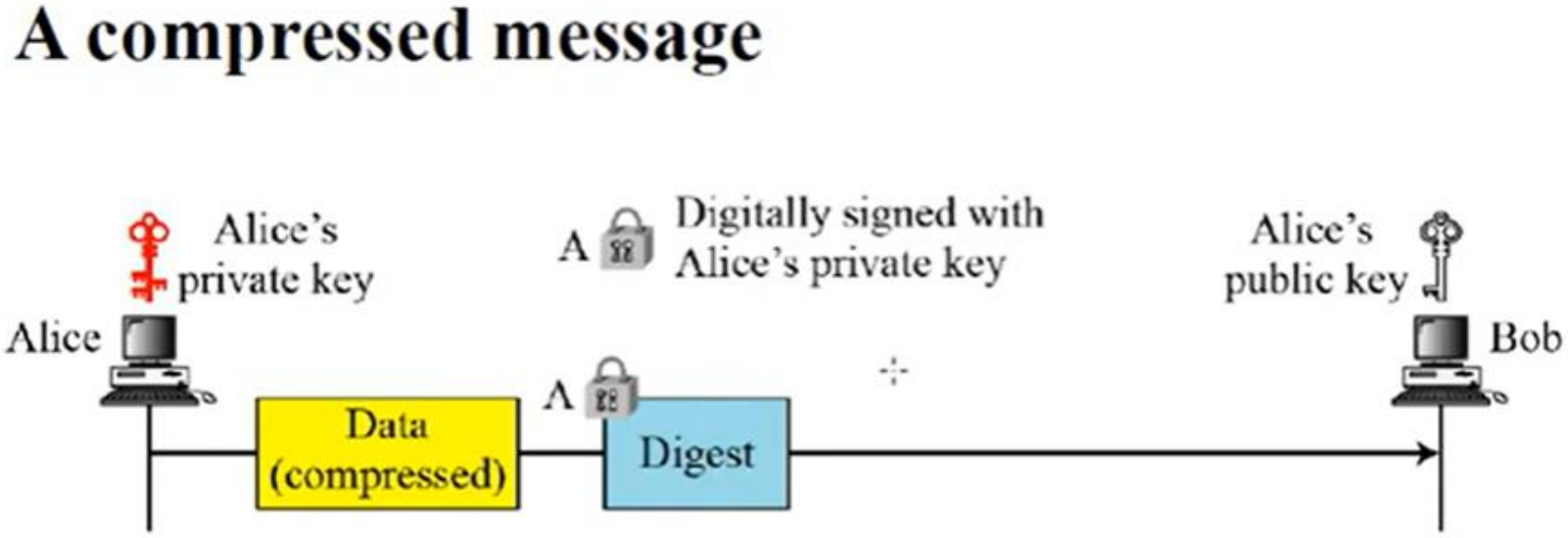
An authenticated message



Unit-5

Service Layer Protocols and Security

Security in Application Layer

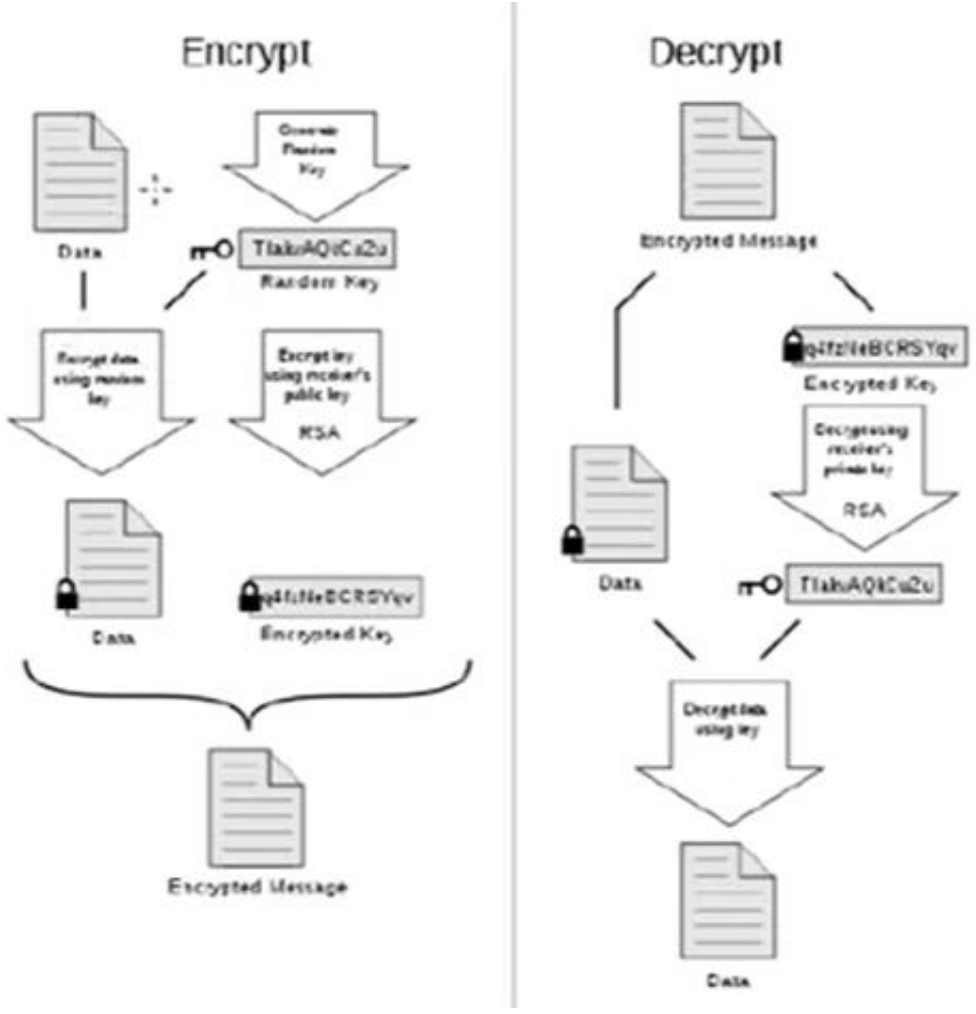


Unit-5

Service Layer Protocols and Security

Security in Application Layer

PGP

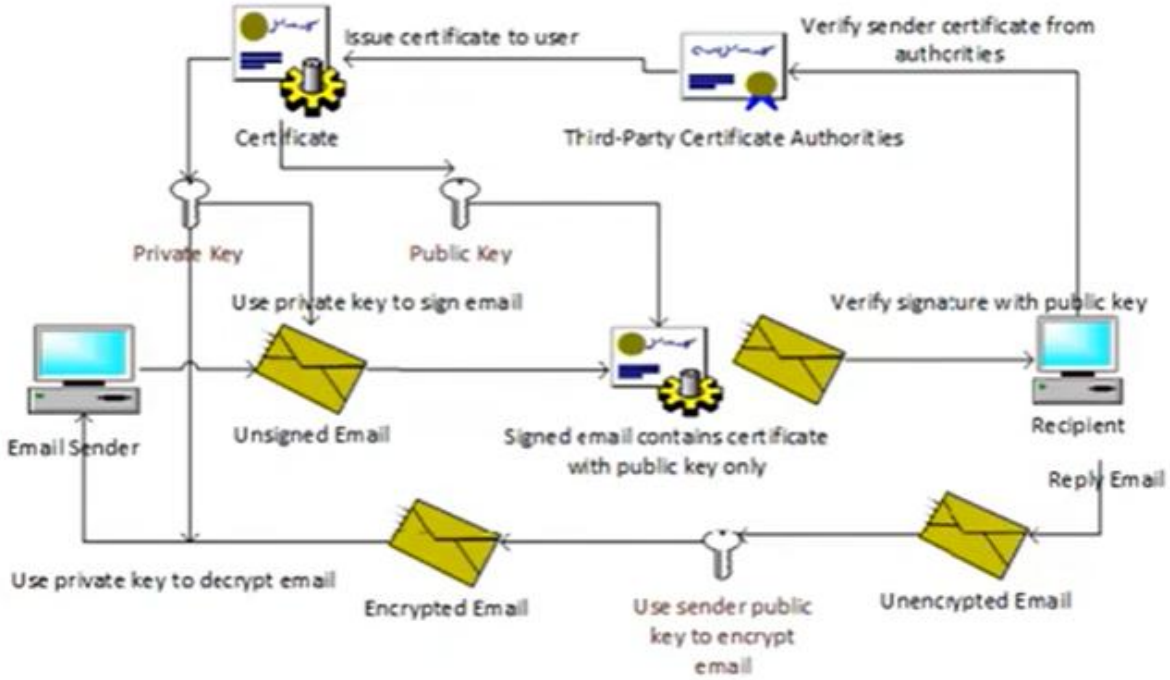


Unit-5

Service Layer Protocols and Security

Security in Application Layer

Secure/Multipurpose Internet Mail Extensions



Unit-5

Service Layer Protocols and Security

Security in Application Layer

S/MIME Benefits

- S/MIME helps to protect users from email-related cyber-attacks such as phishing, spear-phishing, CEO fraud, "whaling," and more. S/MIME email signing certificates prevent malicious users from "eavesdropping" on your email communications

Security Measures for Application Layer Protocols in IoT

Security Requirement	Countermeasure
Confidentiality	Use **encryption** protocols like **TLS/SSL**, **DTLS**, or **AES** to protect data in transit.
Integrity	Implement **Message Authentication Codes (MACs)**, **hash functions**, and **digital signatures**.
Authentication	Use strong **authentication mechanisms** like **OAuth**, **X.509 certificates**, or **JWT**.
Non-repudiation	Apply **digital signatures** and maintain **secure logging** to prevent denial of actions.
Availability	Implement **DDoS protection**, **redundant systems**, and **load balancing** to ensure service uptime.
Access Control	Use **role-based access control (RBAC)**, **OAuth 2.0**, and **API keys** to control access to resources.

Security Concerns in Application Layer Protocols for IoT

Confidentiality:

Problem: Sensitive data transmitted between IoT devices and applications could be intercepted and accessed by unauthorized parties.

Solution: Use encryption techniques to ensure that data is only accessible to authorized users.

Data Integrity:

Problem: Data could be altered or corrupted during transmission, leading to incorrect information being processed or acted upon.

Solution: Use cryptographic hash functions, digital signatures, and Message Authentication Codes (MACs) to ensure that data remains unchanged.

Authentication and Authorization:

Problem: Unauthorized devices or users could gain access to the system, perform malicious activities, or compromise security.

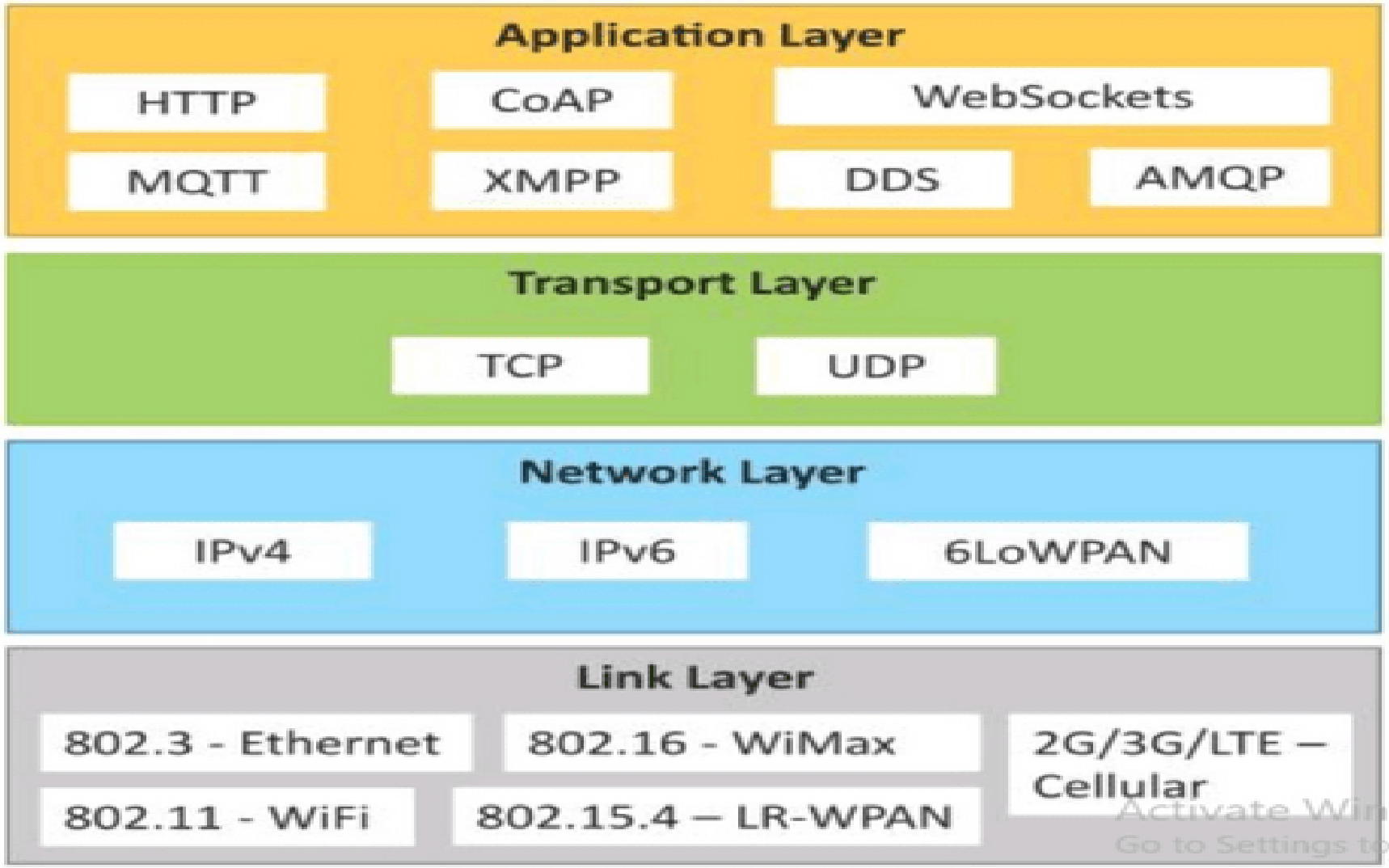
Solution: Employ secure authentication mechanisms (e.g., certificates, tokens) and role-based access control (RBAC) to ensure that only authorized devices and users can access or modify data.

Non-repudiation:

Problem: An IoT device or user may deny performing certain actions or sending specific messages, leading to accountability issues.
Solution: Implement digital signatures and secure logs to provide evidence that actions were performed by specific users or devices.

Availability:

Problem: Attacks such as Denial-of-Service (DoS) can make IoT services or devices unavailable, disrupting system functionality.
Solution: Implement fault tolerance, redundancy, and DDoS (Distributed Denial of Service) protection to maintain the availability of services.


Application Layer
HTTP
CoAP
WebSockets
MQTT
XMPP
DDS
AMQP
Transport Layer
TCP
UDP
Network Layer
IPv4
IPv6
6LoWPAN
Link Layer
802.3 - Ethernet
802.16 - WiMax
2G/3G/LTE – Cellular
802.11 - WiFi
802.15.4 – LR-WPAN

1. CoAP (Constrained Application Protocol)

Description: A lightweight protocol designed for resource-constrained devices in IoT, often used in conjunction with UDP.

Security Considerations:

Encryption: CoAP typically uses DTLS (Datagram Transport Layer Security) for securing communication.
Integrity and Authentication: DTLS provides data integrity, authentication, and protection against eavesdropping and tampering.
Challenges: CoAP is designed to be efficient but can have limitations in terms of providing strong security in constrained environments.

2. MQTT (Message Queuing Telemetry Transport)

Description: A publish-subscribe messaging protocol designed for lightweight, low-bandwidth, and high-latency environments.

Security Considerations:

Authentication: MQTT supports username/password authentication, and more advanced setups can use client certificates for mutual authentication.
Encryption: TLS/SSL is used for encrypting the data payload and ensuring confidentiality.
Data Integrity: TLS provides data integrity and protection from tampering.
Challenges: Although MQTT provides robust security features, it is still susceptible to attacks like message replay or unauthorized access if not configured properly.

3. HTTP/HTTPS (Hypertext Transfer Protocol)

Description: The standard protocol for web-based communication, often used for connecting IoT devices to cloud platforms or web services.

Security Considerations:

Encryption: HTTPS (HTTP over TLS/SSL) ensures the confidentiality and integrity of data.
Authentication: APIs secured with OAuth2 or API keys can authenticate users and devices.
Challenges: HTTP/HTTPS can have overhead and may not be ideal for low-power, resource-constrained IoT devices due to high computational and memory requirements.

4. XMPP (Extensible Messaging and Presence Protocol)

Description: A messaging protocol that allows real-time communication for IoT devices, often used in applications like messaging and presence detection.

Security Considerations:

Encryption: XMPP supports the use of TLS for encrypting data.
Authentication: Uses SASL (Simple Authentication and Security Layer) for authentication.
Challenges: XMPP has extensive features, but can be over-complicated for some IoT applications.

5. AMQP (Advanced Message Queuing Protocol)

Description: A messaging protocol that supports reliable queuing and message routing, used for enterprise and IoT messaging systems.

Security Considerations:

Encryption: AMQP can use TLS for secure communication.
Authentication: Supports strong authentication mechanisms, including certificates.
Challenges: AMQP's complex nature can be an overkill for simpler IoT applications.

6. DDS (Data Distribution Service)

Description: A real-time, publish-subscribe middleware protocol designed for high-performance applications like industrial IoT.

Security Considerations:

Encryption: DDS supports encryption for communication confidentiality.
Authentication and Authorization: Uses access control mechanisms to ensure only authorized devices can participate in data exchange.
Challenges: DDS is often more complex and may not be ideal for low-resource devices.

7. WebSocket

Confidentiality:

Web Socket communication can be intercepted if not properly secured, leading to data leakage.

Solution: Use **WSS (Web Socket Secure)**, which encrypts the communication using **TLS (Transport Layer Security)**, ensuring data privacy and confidentiality during transit.

Integrity:

Data transmitted via Web Socket could be modified or tampered with by attackers during transmission.

Solution: **TLS** ensures that data cannot be modified or tampered with by encrypting the messages and providing data integrity checks.

Authentication:

Web Socket connections often lack built-in authentication mechanisms, which could allow unauthorized clients to establish connections.

Solution: Secure Web Socket connections by implementing **authentication mechanisms** (e.g., **OAuth2**, **API tokens**, or **JWT** tokens) during the handshake process to ensure only legitimate clients can connect.

Cross-Site Web Socket Hijacking (CSWSH):

Malicious websites could trick a user's browser into opening a Web Socket connection to an unintended or vulnerable server.

Solution: Always verify the **Origin** header in WebSocket handshakes to ensure that connections are coming from trusted and authorized sources.

Denial of Service (DoS):

Attackers could flood the WebSocket server with excessive connections or data, leading to a service outage.

Solution: Implement **rate-limiting**, **connection throttling**, and **firewall** rules to limit the number of incoming connections per client and avoid service degradation.

Session Hijacking:

If a WebSocket connection is not properly authenticated, attackers can hijack a session to gain unauthorized access.

Solution: Use secure session management techniques, such as **token-based authentication** and **two-factor authentication (2FA)**, to ensure that only authorized clients can maintain an active WebSocket session.

Replay Attacks:

An attacker could intercept and replay valid WebSocket messages to the server, causing unintended or malicious behavior.

Solution: Implement mechanisms like **timestamps**, **nonce values**, or **message sequencing** to prevent replaying old messages